Spring Boot Debug REPL and MCP

Detailed user manual - 0.25.0

Tabs, buttons, fields and workflows for the IntelliJ IDEA plugin. Experiment with Java inside a running application, inspect objects, save snapshots, reload code and work with Claude.

Who is this for? Start with chapter 1 if you are enabling the REPL for the first time. To find a particular button, use the contents or PDF bookmarks to open the relevant tab's chapter. Chapter 24 covers shortcuts; chapters 25 and 49 list the MCP tools.

What does the REPL provide? Work with the running Java application's own classes and Spring beans. The session retains imports, variables and methods. Inspect a result repeatedly, then save the data you need as a DATA snapshot.

Scope: the three main tabs (Workbook, MCP, Tools), all ten supporting Tools panels, subtabs, buttons and settings wired into the 0.25.0 source. Labels match the plugin's English UI. IDEA themes, keymaps and window sizes may change their placement. The layout diagram is a schematic, not a live screenshot.

Updated: October 2, 2026. This full English edition covers the same features as the Hungarian manual, including Claude/MCP setup and the SQL/Hibernate recording tools.

In the IDE: Code > Spring Boot Debug REPL and MCP > Help (PDF) > English or Magyar. The Java editor context menu offers the same choices. Workbook > Tools > Help (PDF) opens a language picker. Both manuals are bundled and work offline.

3
main tabs, all tools retained

82
MCP tools for Claude

200 MiB
default DATA snapshot limit

Start — **Experiment** — **Snapshot** — **Verify**

Contents

Click a chapter title to jump to its page. The PDF reader's bookmarks show the same structure.

01. First launch and profiles . . . 4
02. Map of the interface . . . 5
03. Workbook: connection and execution . . . 7
04. Workbook: workbook controls . . . 8
05. Results, JSON and code checking . . . 9
06. Variables: session values . . . 10
07. Snapshots: what to save . . . 11
08. Snapshots > Saved: controls and loading . . . 12
09. Snapshots > Capture next . . . 13
10. Snapshots > Compare . . . 15
11. Inspector: browsing objects . . . 16
12. Tap / Trace: observing a running app . . . 17
13. Cases / Reload: creating a case . . . 18
14. Cases / Reload: execution and HotSwap . . . 19
15. Debugger: local values in the REPL . . . 20
16. HTTP: composing a saved request . . . 21
17. HTTP: buttons and execution . . . 22
18. AI: from prompt to Java suggestion . . . 23
19. Imports and plugin settings . . . 24
20. MCP: local server controls . . . 25
21. Claude Code: connection and first task . . . 27
22. Claude Desktop and another machine . . . 28
23. Complete workflow: capture to fix . . . 29
24. Keyboard shortcuts and menus . . . 30
25. MCP tool reference for Claude . . . 31
26. Troubleshooting from the UI . . . 35
27. Limits, terminology and references . . . 36
28. Execution modes and transactions . . . 37
29. Reproduction in one bundle . . . 38
30. Expected exceptions and capture metadata . . . 39
31. Audit, secrets and future work . . . 40
32. CASE 2.0: fields and assertions . . . 41
33. Parameters, lifecycle and results . . . 43
34. Export a CASE as JUnit 5 . . . 44
35. Notebook: cell state and dependencies . . . 45
36. Saving and opening a workspace . . . 47
37. Snapshot versions and provenance . . . 48
38. Snapshot points in Java source . . . 49
39. Recorded call trees and source-side values . . . 50
40. AI-agent access to call recordings . . . 55
41. SQL, N+1 alerts and calls down to the DB . . . 57
42. Hibernate in recorded call flows . . . 60
43. Recorded call to runnable CASE . . . 63
44. Edit DATA copies and create variants . . . 64

45. Bean Explorer 65
46. Watches and field changes 66
47. Async flow and transaction boundaries 67
48. HotSwap and affected CASE review 68
49. MCP workflow tools, events and tasks 69
50. Data handling and font licenses 71

01. First launch and profiles

Goal: start your Spring Boot application successfully and make `ctx` available in the REPL.

1. Install `build/distributions/sb-repl-0.25.0.zip` through Settings > Plugins > Install Plugin from Disk. Restart IDEA after updating the plugin.
2. Open your usual Spring Boot Run Configuration. Select **Enable Spring Boot Debug REPL and MCP**, keeping the normal configuration's main class, environment and VM options.
3. For a Spring Boot configuration, enter profiles such as `dev,llm-openai` in **Active profiles**. For a plain Application configuration, use this program argument:

```
--spring.profiles.active=dev,llm-openai
```

4. Start the application with **Run** or **Debug**. Debugger features require Debug; ordinary REPL use works with Run.
5. Open the **Spring Boot Debug REPL and MCP** tool window and wait for **READY**. In Workbook, run:

```
ctx.getBeanDefinitionCount()
```

Expected result: a number in the Value panel. This confirms that the expression reached the bound Spring context; it does not test the application's business behavior.

Which launch mode should I use?

Mode	When to use it
Normal Spring Boot + REPL checkbox	Recommended for everyday work; retains Spring profiles and environment settings.
Application + REPL checkbox	Launch a Java main class; supply Spring profiles through arguments or the environment.
Tools > Attach & Inject Dev Runtime	Attach to an already running JVM you select. The bridge can help discover the context after late attachment.
Legacy separate Spring Boot Debug REPL and MCP configuration	Retained for compatibility; prefer a normal configuration for new setups.

The checkbox uses the plugin's bundled agent; no separate application dependency is required. Calls to `SnapshotHelper` in application source do require the bridge. A bare `dev,llm-openai` argument does not activate profiles. A Spring configuration error after nREPL has started must still be fixed in the application.

Connection states

DISCONNECTED: no connection. **CONNECTING:** establishing a connection. **SESSION_READY:** Java session available, though Spring may not be ready. **WAITING_CONTEXT:** waiting for a context. **READY:** Spring bound. **FAILED:** connection setup failed; read the detailed status and application log. During execution, **Queued / running** may also appear.

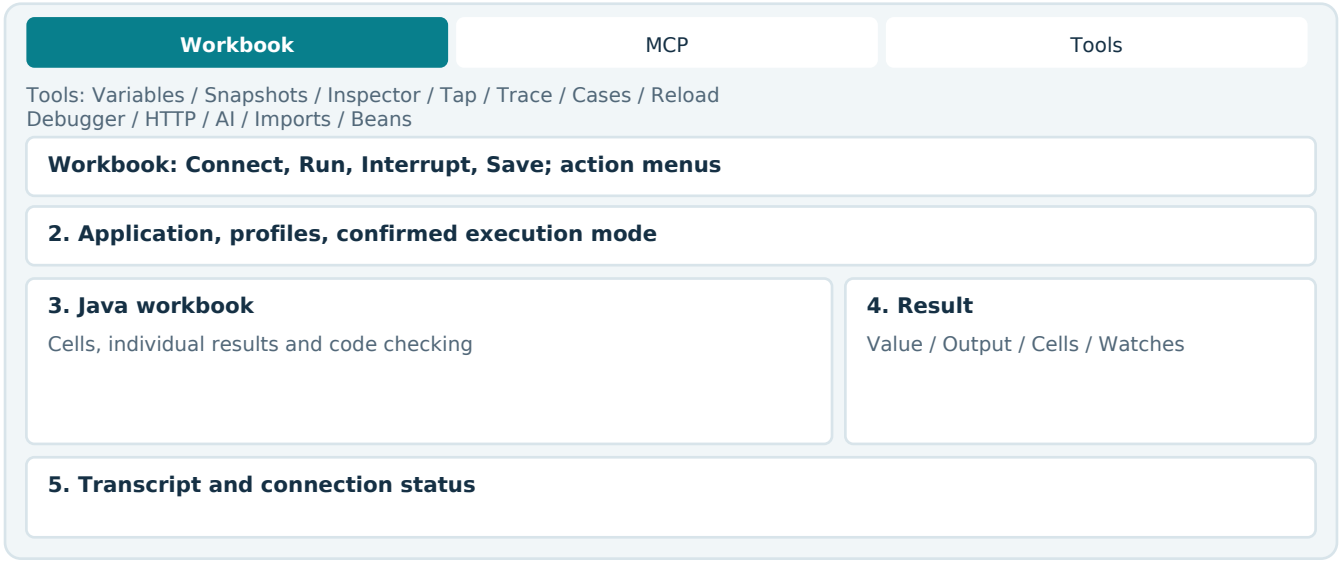
02. Map of the interface

The three main tabs are **Workbook**, **MCP** and **Tools**. Start with Workbook for Java cells or MCP for an AI client. Tools lists the supporting panels on its left. No feature or MCP tool has been removed.

Main tab	Purpose
Workbook	Connection, Java cells, execution, checking and results.
MCP	Local MCP server, client configuration and permissions. Advanced settings are collapsed until needed.
Tools	Variables, snapshots, inspection and the other supporting workflows below.

Tools panel	Purpose
Variables	List, insert and inspect variables in your IDE session.
Snapshots	Saved, Capture next, Compare and Versions for persistent data and capture rules.
Inspector	Navigate real objects through Value and Fields.
Tap / Trace	Observe application values and method calls.
Cases / Reload	Save snapshot-based cases, define expected results, rerun and HotSwap.
Debugger	Control IDEA's current debug session and transfer local values.
HTTP	Compose and run saved HTTP requests; generate snippets.
AI	Review a prompt, send it to the configured API and obtain code suggestions.
Imports	Save imports and apply them to a session.
Beans	Bean definitions, dependencies, proxy metadata and prepared calls.

The following chapters use the panel names. For example, **Snapshots > Saved** means **Tools > Snapshots > Saved**. **Workbook > Tools** is the workbook's action menu, separate from the main Tools tab. Workbook keeps four primary icons and four action menus; drag the divider between code and results to resize them.



Result subtabs: Watches holds pinned before/after observations. Workbook > Value > Tree / Formatted / Raw. Cells shows individual execution evidence and output. **Output / errors** contains stdout/stderr and errors. **Inspector > Value** uses the same structured viewer; **Inspector > Fields** navigates the actual object.

03. Workbook: connection and execution

Location: Workbook's top toolbar. The four primary icons are **Connect**, **Run cell / selection**, **Interrupt** and **Save workspace**. Hover to see a name. The adjacent **Run**, **Workspace**, **Session** and **Tools** menus contain detailed actions. Connect uses a known endpoint or one selected in Settings; it does not start Spring.

Where to look: Run contains cell execution and dependency analysis. Workspace contains workbook files, saving and history. Session contains connection, Bind, Reset and Execution settings. Tools contains checking, beans, imports, reproduction, audit, PDF help and MCP. Detailed errors and the transcript remain below; application/profile and confirmed execution mode appear above.

Button / control	Behavior and requirements
Connect	Connect to the known agent endpoint. Reuses an existing live connection without creating another session.
Open endpoint	Select a local `.properties` endpoint file and connect to its agent. Do not select application.yml.
Disconnect	Close the REPL connection and its session. Spring keeps running. MCP access also stops.
Bind	Try binding a Spring context that has become ready. Success means READY; a context change requires Reset.
Run cell / selection	Execute selected text, or the current cell when nothing is selected.
Run + next	Execute, then move to the next existing cell after success if the text and cursor have not changed.
Run all	Execute nonempty cells sequentially with separate results. Stops on errors, edits or interruption. Earlier cells' effects may happen again.
Interrupt	Request interruption of your running evaluation. Cooperative cancellation, not rollback.
Reset	After confirmation, discard declarations, handles and LIVE pins, then build a new session state for the current context. DATA survives.
Complete	Request completion at the cursor without evaluating code.
Tools > Toggle live check	Enable or disable automatic checking without execution. Enabled by default.
Check code	Analyze the workbook immediately.
Help (PDF)	Choose the English or Hungarian offline manual. No REPL connection is needed.
MCP	Switch to MCP; this alone does not start the server.

04. Workbook: workbook controls

Location: Workspace menu. Insert bean and Apply configured imports are in Tools. Source text and live session state can differ; chapters 35 and 36 explain the state indicators and workspace files.

Button	Behavior
Insert cell	Add a new `// %%` boundary at the end of the current cell and move the cursor there.
Open workbook	Load a UTF-8 workbook of up to 1,000,000 bytes, replacing the editor text. Does not run code or reset the session.
Save workbook	Save source as a `.jsh` file. Does not save live objects.
Save RECIPE	Save selected code, or the whole workbook without a selection, under a name in runtime storage. Does not execute it.
History	Choose from up to 100 recent snippets and insert the selected code at the cursor.
Clear history	Clear history and the lower Workbook transcript, leaving variables intact.
Insert bean	Choose a bean name/type and optional public type; insert a `ctx.getBean(...)` declaration. It retrieves the bean only when run.
Apply configured imports	Import enabled fully qualified class names from Imports into this session.

Try a two-cell workbook

```
// %% Preparation
import java.util.List;
var numbers = List.of(2, 4, 6);

// %% Experiment
numbers.stream().mapToInt(Integer::intValue).sum()
```

Run the first cell, then the second. The second produces `12`. Editing only the second cell does not require recreating its input.

Cell boundaries: put `// %%` on its own line, optionally followed by a title. Text inside a string, text block or block comment does not split the workbook. A file without boundaries is one cell.

Persistence: Save workbook / Save RECIPE preserve code. DATA preserves values. Opening a workbook after reconnecting does not recreate its variables automatically; run the required preparation cells.

05. Results, JSON and code checking

Location: the result panel on the right of Workbook and the checking status below the editor.

Control	Display or action
Value > Tree	Collapsible object fields, maps, lists, arrays, types, nulls and repeated references.
Value > Formatted	Readable indented text; JSON uses two-space indentation.
Value > Raw	Available raw result text, including the original JSON string. May still be a bounded preview.
Expand / Collapse	Expand or collapse the displayed tree; does not fetch new object data to arbitrary depth.
Copy formatted	Copy the formatted display to the clipboard.
Output / errors	Evaluation stdout/stderr and errors.
Inspect result	Open the object from the last usable result handle without rerunning the expression.
Pin LIVE	Save a live reference to the current result under a name.
Freeze DATA	Persist the current result; asks for a name and optional declared type.

Automatic JSON detection: a valid JSON object or array string becomes a tree and an indented view. Invalid JSON remains text. **Partial preview** means a display limit was reached, not that the original object is incomplete.

Checking and completion

Live check: analyzes the workbook after 650 ms without typing. Errors and warnings are underlined; hover to read them. Declarations in earlier cells provide analysis context even before execution. Initializers are not called and the live session is not modified.

Complete / Ctrl+Space: suggests completions from the current JShell session; it can also request completion 300 ms after typing an identifier or dot. Execute the declaration cell first. The checker may therefore recognize a variable that live-session completion does not yet know.

This is compiler syntax/type analysis, not full business validation or a comprehensive linter. `last1`, `last2` and `last3` hold recent result objects; `lastError` holds the last evaluation error.

06. Variables: session values

Location: Tools > Variables. It lists variables acknowledged by the runtime for the IDE session and refreshes when opened.

Button	Usage and effect
Refresh	Reload variables; clear the view when disconnected.
Insert	Insert the selected variable's name at the Workbook cursor and switch to the workbook. Does not execute code.
Inspect	Open the selected variable's object in Inspector.
Previous / Next	In the current main-window wiring these also open Inspector for the selected variable. Use Inspector's own paging buttons for actual pagination.
Drop	Drop the selected Java declaration and refresh. Does not delete database data or saved DATA snapshots.

Choosing an action

1. You already retrieved a large list named `items`: use **Variables > items > Inspect** to examine it without querying again.
2. To reuse it in another cell, choose **Insert**, complete an expression such as `items.size()`, then run it.
3. To remove an experiment's unused variable, choose **Drop**. Other objects or beans may still reference the value; immediate memory release is not guaranteed.

What is outside this list?

It is not a list of every application local or every Spring bean. For a method-local value use **Debugger > Capture to REPL** or an application capture/tap point. Choose beans through **Workbook > Insert bean**.

Claude's separate MCP-session variables do not appear here. Save DATA in one session and load it in the other to transfer a value between IDEA and Claude.

07. Snapshots: what to save

Location: Snapshots. Choose a mode based on what must be preserved and for how long.

Mode	Preserved content
LIVE	Original object reference in this session, for up to 30 minutes. Later mutations remain visible. Lost when the session or context closes.
DATA	Selected data in a versioned JSON envelope. Persists in a file; loading may construct a new object.
RECIPE	Java source, such as preparation cells. Loading inserts it; Run executes it.
CASE	Input and expected DATA names, Java code and input type settings. Run it in Cases / Reload.

Data, type and size

DATA requires Jackson in the application. Saving uses a separate copy of an existing ObjectMapper or a separate mapper; it does not modify the application's mapper. Saving may invoke getters; loading may invoke constructors and deserialization code.

Save DTOs, records or focused projections. Select the data needed for reproduction instead of saving entire Spring beans, connections, Hibernate proxies or the whole application graph. Classes declared only in JShell may be unavailable in another session or after restart.

The default file limit is **200 MiB**, including metadata. Loading large data may require more heap than this. Use **Import file / Export file** for files and **Paste JSON** for small inline JSON. IDE inline import is limited to 2 MiB; MCP has a smaller limit.

Collections and mixins

In **Freeze DATA**, supply a generic element type such as `java.util.List<com.example.ItemDto>` to retain element types on restore.

```
import com.baader.devrt.SnapshotManager;
SnapshotManager.save("item-list", items,
    "java.util.List<com.example.ItemDto>");
```

Replace `com.example` names with application classes. Add a snapshot-specific Jackson mixin with `SnapshotManager.addMixIn(Target.class, Mixin.class)`. It changes this session's snapshot serialization. Omitted data cannot be recovered by loading the snapshot later.

DATA is not a full application backup. It does not save call stacks, transactions, databases or open connections, and loading it does not restore Spring bean state.

08. Snapshots > Saved: controls and loading

Location: Snapshots > Saved. Each row shows its name, `[LIVE]`, `[DATA]`, `[RECIPE]` or `[CASE]` mode, and type. Ctrl/Cmd-click selects multiple rows.

Button	Purpose and required input
Refresh	Refresh available snapshots.
Pin LIVE	Ask for a snapshot name and existing Java variable. A blank variable uses the last result.
Freeze DATA	Persist a value with an optional fully qualified Java type. Supply an existing variable, not an expression.
Load	Load selected DATA/LIVE into a REPL variable, default `restored`. For RECIPE, only insert its source.
Info	Show type, mode, time and size information.
Paste JSON	Ask for a name and JSON text, then import as DATA without evaluating Java.
Import file	Select a local JSON file and snapshot name; the runtime imports the file.
Export file	Export a selected persistent snapshot. Confirm overwriting an existing file. Freeze LIVE to DATA first.
Compare	Select exactly two DATA snapshots to open Compare and compare them.
Delete	Delete the selected snapshot after confirmation. Other sessions may use shared DATA.

Example: reusable input

1. Create an `input` variable in Workbook or capture one from the debugger.
2. **Saved > Freeze DATA**: name `cv-input-42`, variable `input`, type your DTO's full name. Accept an empty optional type with OK, not Cancel.
3. Later, select it and choose **Load**, using variable name `restoredInput`.
4. Inspect it with **Variables > Refresh > restoredInput > Inspect**.

Load uses the stored DATA type without asking for a type override. Use the appropriate runtime/MCP loading API to load a different type. Open a CASE through **Cases / Reload > Load selected**.

Local files: the running application uses Import file / Export file paths, so it must see the same filesystem as the IDE. Saving under the same name creates a new current version; earlier values remain under Versions. Deleting a snapshot removes its versions too.

09. Snapshots > Capture next

Goal: automatically save a selected value during a subsequent real application call. Arming a rule does not trigger an HTTP request or business operation.

Field / button	Meaning
Capture point	Exact name of the point in application code, for example `cv-input`.
Snapshot name (creates a new version)	DATA name; saving an existing name creates another persistent version.
Exact case ID filter (empty = any)	Exact filter for the capture call's `caseId` argument. Blank accepts any ID. This is not a saved CASE name.
Declared type (optional)	Full Java type, optionally generic, for the saved data.
Arm capture rule · 5 minutes	Arm a new independent rule for five minutes; it stops after the requested number of saves.
Disarm selected rule	Cancel your selected waiting rule. Does not interrupt serialization already in progress.
Capture count	1-100 saves. Multiple captures get a sequence number, or use `${sequence}` in the name.
Capture every Nth matching call	1-10000; save the first match and every Nth subsequent match.
Refresh status	Fetch status and your rules immediately. Click a row to select a rule.

Prepare application code

The bridge must be available when compiling source. In the plugin repository, install it to your local Maven repository:

```
mvn -f sb-repl-bridge/pom.xml install -Dgpg.skip=true
```

Use development dependency `hu.baader:sb-repl-bridge:0.25.0`. The checkbox's agent alone does not provide source-level imports. In the processing method:

```
import com.baader.sbrepl.bridge.SnapshotHelper;
SnapshotHelper.capture("cv-input", requestId, inputDto);
```

`requestId` is a String; `inputDto` is the value to save. For an expensive projection use `captureLazy("cv-input", requestId, () -> projectToDto(inputDto))`. The supplier runs once only for an armed matching call. Saving is synchronous on the calling thread.

Reading capture status

ARMED: waiting. **CAPTURING:** a call has claimed the save. **SAVED:** success with size and duration. **FAILED:** failed with details. **EXPIRED:** expired. **CANCELLED:** cancelled.
OTHER_SESSION: you have no rule, but another session has an active one. A JVM supports up to 16 independent active rules. A rule skips further matches while saving; overlapping rules share one supplier evaluation per application call. A failing rule stops.

After success, the snapshot appears in Saved. Without an armed rule or agent, capture returns `false`; plain `SnapshotHelper.save(...)` saves unconditionally and can throw.

10. Snapshots > Compare

Open: select exactly two DATA snapshots in Saved, then **Compare**. Check their Before/After order in the title.

Control	Behavior
Before / After title	Names of the two compared snapshots.
Field / array index	Path to the differing field or array item.
Change	`ADDED`, `REMOVED` or `CHANGED`.
Before / After columns	Bounded previews of differing values.
Previous / Next	Page through 100 differences at a time. Next is enabled only when more results exist.
Swap before / after	Reverse the comparison and restart at the first page.

Interpreting results

No DATA differences: no content differences were found in the completed examination. Save timestamps and envelope metadata do not count. A missing field and `null` are distinct.

Partial comparison: traversal reached a limit or was interrupted. Reported differences are useful, but zero matches does not prove equality. Value previews can be shortened.

Suggested workflow

1. Before a fix, save the result as `cv-before`.
2. After the fix, run the same input and save `cv-after` DATA.
3. Select both and click Compare.
4. Check intended changes and unexpected added or missing fields.

Comparison works on saved JSON; it does not restore DTOs or rerun a service call. Cells contain previews. Load the relevant snapshot and use Inspector for deeper examination.

11. Inspector: browsing objects

Open from: Workbook > Inspect result, Variables > Inspect, Tap / Trace > Inspect selected, debugger capture or CASE > Inspect actual. Inspector attaches to the selected live object.

Button / field	Behavior
Value	Tree / Formatted / Raw preview of the current object.
Fields	Table with Field / index, Type, Preview and Access.
Open selected / double-click	Navigate into a selected field or element. Explain an Access error when it cannot be opened.
Back	Return to the previous object.
Refresh	Read the current object's field page again.
Previous / Next	Page through 50 elements at a time. Large containers have a traversal limit.
Variable	Target variable for Bind current; default `inspected`.
Optional Java type	Optional public/declared type for binding and DATA saving.
Bind current	Bind the currently opened real object to the chosen REPL variable.
Snapshot	Name to save; default `inspected-data`.
Pin LIVE / Freeze DATA	Save a live pin or persistent DATA from the currently opened object.

Navigation matters: expanding Value changes the display only. **Fields > Open selected** changes the target of **Bind current** and saving. Navigate into the desired object before binding it.

Example: extract one response item

1. Open a service-call result in Inspector.
2. In Fields, select `items` and choose Open selected.
3. Page, select an item and choose Open selected again.
4. Enter Variable `selectedItem`, then Bind current.
5. Use `selectedItem` in Workbook, or Freeze DATA for a persistent sample.

Automatic previews read direct fields without rerunning the original expression. Explicit navigation can execute access code, such as a custom collection implementation. After Reset or a context change, open a new value; old handles are invalid.

12. Tap / Trace: observing a running app

Tap: selected application locations publish values. **Trace:** follow calls to a named method. Both retain live references briefly.

These controls are in **Live events**. **Recorded calls** builds a call tree and saveable value recording from actual method calls in selected classes, with source-side viewing; see chapter 39.

Field / button	Effect
Exact tap label (empty = all)	Blank accepts all taps; otherwise requires the exact label. Applied by Start tap.
Start tap / Stop tap	Start or stop tap reception for your session.
Refresh	Refresh events and enabled traces.
Clear values	Clear retained event values. Stop tap reception and trace configurations separately.
Inspect selected / double-click	Open the event's value in Inspector.
Class	Fully qualified application class to trace.
Method (all overloads)	Method name, including declared overloads with the same name.
Trace / Untrace	Enable or disable that method's trace.
Stop all traces	Stop all traces owned by your session.

Columns are **ID**, **Time**, **Kind**, **Label / method**, **ms** and **Preview**. Selecting a row in the lower trace list fills Class/Method. While visible, the events panel also refreshes automatically.

Add taps to your code

```
SnapshotHelper.tap("cv-input", inputDto);
SnapshotHelper.tap("cv-result", resultDto);
```

The bridge dependency is required. Without the agent or subscribers nothing is saved. Tap alone does not serialize values or create DATA files.

Trace without changing source

Place the cursor on a method in normal Java source and choose **Code > Spring Boot Debug REPL and MCP > Trace Method**, or use Class/Method on this tab. Arguments, return values, exceptions, threads and duration can be examined.

A session retains up to **128 event values** for **five minutes**. The `dropped` counter may indicate missing events; this is not a complete audit log. Save important values as DATA through Inspector and stop observation when no longer needed.

13. Cases / Reload: creating a case

Chapters 32-34 cover the assertion, parameterization and JUnit-export fields introduced in 0.15. The simple CASE format below remains supported.

Goal: rerun recorded input and compare the result with expected DATA. A CASE is a development experiment in a running application; it does not replace project tests.

Field / button	Meaning
Case name	Name of the saved case.
Input DATA	Input snapshot; the list offers DATA entries.
Expected DATA	Expected snapshot for comparison.
Variable	Input name in the case's own evaluator; default `input`.
Optional input type	Optional fully qualified input Java type.
Java code field	Code to run on the input. The final produced value is compared with expected DATA.
Refresh	Refresh cases and available DATA names.
Save case	Save current fields and code without running.
Load selected	Load the selected definition for editing without running.

A small standalone example

Create input and expected values in Workbook, then Freeze DATA for each:

```
var numbers = new java.util.ArrayList<Integer>(
    java.util.List.of(1, 2, 3));
int expectedSum = 6;
```

Input: `demo-input`, type `java.util.List<java.lang.Integer>`. Expected: `demo-expected`, variable `expectedSum`. In Cases / Reload set:

- Case name: `demo-sum`; Input DATA: `demo-input`; Expected DATA: `demo-expected`.
- Variable: `input`; Optional input type: `java.util.List<java.lang.Integer>`.
- Code:

```
input.stream().mapToInt(Integer::intValue).sum()
```

Click **Save case**, select it and click **Run saved cases**. The expected status is `PASSED`.

A CASE runs in its own fresh evaluator. A workbook variable named `processor` is not transferred automatically; retrieve the service with `ctx.getBean(...)` inside the case if needed.

14. Cases / Reload: execution and HotSwap

Location: the same tab's controls and result table. Select 1-20 saved cases per run.

Button	Effect
Run saved cases	Run selected saved definitions sequentially. Save form edits first with Save case.
Rerun failed	Rerun cases previously reported as FAILED, ERROR or INCONCLUSIVE.
Stop	Stop further workflow steps; may request interruption of current evaluation. Does not undo external effects.
Inspect actual	Open the selected case's latest actual result in Inspector, if its event value is available.
Use open Java editor	Select normal editor `.java` source for reload, including current unsaved content.
Choose Java file	Choose another `.java` file; each run reads its latest content.
Reload + run selected	Compile and request HotSwap; run selected cases only after a successful update.

Result	Meaning
NOT RUN	No execution result yet.
PASSED	Actual value matches expected DATA.
FAILED	Content differs; inspect detailed output.
ERROR / CANCELLED	Error or cancelled execution.
INCONCLUSIVE	Could not decide, for example because a comparison limit was reached. Not a passing test.

What does preserving state during reload mean?

After a method-body change supported by the standard JVM, existing application objects remain. New fields, methods or structural changes may require restart. HotSwap is **manual**; saving source alone does not update the JVM.

Reload Class in normal Java source performs only the update. **Reload + run selected** also runs the selected CASEs. After an error, do not assume the new code is running.

Despite a fresh evaluator, a CASE accesses real Spring beans. Database writes, HTTP calls and bean state changes may persist after execution.

15. Debugger: local values in the REPL

Requirements: run the application in Debug mode with the REPL checkbox enabled. The tab uses IDEA's current debug session and selected stack frame.

Button / field	Behavior
Pause	Request suspension of the current debug session.
Resume	Continue the suspended program.
Step over / Step into / Step out	IDEA's normal stepping operations in a suspended session.
Show stack / locals	Open IDEA's Debug window and execution point. Select the frame there.
Java expression	Expression in the selected frame, such as `input` or `this`.
Evaluate in frame	Evaluate and display the expression in that suspended frame.
REPL variable	Name for the transferred value; default `debugValue`.
Capture to REPL	Evaluate once and retain the object for transfer into the REPL after Resume.

Capture a service's local input

1. Set a breakpoint where `inputDto` is available.
2. Trigger the application request; the debugger stops.
3. Use **Show stack / locals** to select the correct frame.
4. Expression: `inputDto`. REPL variable: `capturedInput`.
5. Click **Capture to REPL** and wait for "Resume to import".
6. Click **Resume**. Inspector opens with the transferred value.
7. Work with `capturedInput`, or **Freeze DATA** for stable input.

Capture verifies that the target JVM and REPL session match. It does not resume the application automatically; pending transfer expires after five minutes. This is the same object, not an automatic copy. The resumed program may mutate it, so preserve an unchanged sample as DATA.

16. HTTP: composing a saved request

Location: Tools > HTTP. Saved requests are on the left, details on the right and the HTTP console below. Its client sends requests from the IDE process; this alone does not require a JShell connection. Some HTTP form labels are Hungarian in the current UI; their meanings are given below.

Field	Purpose
Case ID	Identifier for the panel and generated helper. Not the same as a snapshot CASE.
Név (Name)	Human-readable request name in the list.
Fő téma / Verzió / Leírás	Topic, version and description metadata for organizing requests.
HTTP	Method selector and URL. Supports GET, POST, PUT, PATCH, DELETE and HEAD.
Headerek (Headers)	Editable Header / Érték (value) rows, such as Content-Type and Authorization.
JSON Body	Request body text. The label does not imply automatic JSON validation.

Example development request

1. Use **+ / Add** above the left list to create a request.
2. Give it a name and your application's full URL.
3. Select a method; add `Content-Type: application/json` for JSON.
4. Paste the input into JSON Body.
5. Click **Mentés (Save)**, then separately **Play**. Status and response appear below.

The suggested `http://localhost:8080/api` is an initial value, not a guaranteed endpoint. Use your application's actual route and port.

Substitution and storage

`${ENV_NAME}` placeholders work in URLs, headers and bodies. **Play** uses the IDEA process environment. A generated snippet run in JShell sees the application JVM's environment instead. Run Configuration environment variables are therefore not necessarily visible to the HTTP panel.

Sensitive URL, header and body values are stored in PasswordSafe when saved by the panel. Literals inserted into a workbook remain in saved `.jsh` or RECIPE source.

17. HTTP: buttons and execution

Icon names may appear as tooltips. **Double-clicking a request runs it**, unlike the editing workflow for snapshots or CASEs.

Button / gesture	Effect
+ / Add	Create a request definition with initial values. Does not send a request.
- / Remove	Delete the selected definition after confirmation. Does not send HTTP DELETE.
Duplicate	Copy the selected request with a new ID. Save edits before duplicating.
Run Selected	Send a request using the current fields.
Double-click a list item	Run that request.
Header hozzáadása	Add a blank header row.
Header törlése	Delete the selected header, or the last row if none is selected.
Mentés	Save the current form.
Play	Save and send the current request.
httpReq.perform snippet	Insert a helper for saved requests and a call for the selected request into Workbook. Only Run there sends the request.

Changing requests and several other panel actions also save current fields. Use Mentés as an explicit review point. The snippet is generated from the request set; review the inserted code before execution.

Reading the console

The console shows method, path, status code, duration, Content-Type and a shortened response. A 2xx status means HTTP success; check business meaning in the body separately. This console is separate from Workbook's Value / JSON tree.

Connection timeout: **10 seconds**. Request timeout: **30 seconds**. Request and response bodies are limited to **1 MiB**; the console preview is at most 65,536 characters. Rerunning the same request ID requests cancellation of the earlier client-side call without undoing server-side effects.

Connect it to capture: first choose Snapshots > Capture next > Arm capture rule, then Play the matching HTTP request. Verify capture status **SAVED**; HTTP 200 alone does not prove a snapshot was created.

18. AI: from prompt to Java suggestion

Location: Tools > AI. This uses the plugin's own API client. Configure external Claude access separately in MCP.

Field / button	Behavior
Request	Describe the Java snippet you want.
Exact prompt sent to the configured API	Full editable prompt used when sending.
Generated Java code	Received response, ready to insert into the REPL.
Refresh bean metadata	Fetch bean names/types and refresh metadata for prompt preparation.
Prepare prompt	Compose a prompt from the request and metadata for up to 500 beans. Does not contact the API.
Send reviewed prompt	Send the middle field's current text to the API configured in Settings.
Insert response into REPL	Insert the response at the workbook cursor and switch there. Does not execute it.

Suggested sequence

1. In Settings, choose provider, model and URL, and supply the required API key.
2. Click **Refresh bean metadata** and fill Request.
3. Click **Prepare prompt**, review the full middle field and edit if necessary.
4. Click **Send reviewed prompt**, then **Insert response into REPL** after receiving a response.
5. In the workbook, choose **Check code**, then separately **Run cell / selection**.

After changing Request or bean metadata, use Prepare prompt again; Send does not automatically rebuild it. The prompt limit is 100,000 characters.

AI or MCP?

AI panel: ask for code suggestions, then insert and execute them yourself. Opening the panel sends no data to the provider.

MCP: external Claude uses your task and permitted tools to inspect status, check code, execute, inspect objects and manage snapshots. It receives its own REPL session.

The Settings value **mcp-offline** disables the built-in AI panel. Despite its name, it does not start an MCP server; use **MCP > Start MCP**.

19. Imports and plugin settings

Imports tab

Columns: **On**, **Alias**, **Fully Qualified Name**. **+ / Add** creates a row, **Edit** edits the selection and **- / Remove** deletes it. On selects imports to apply.

Add Import / Edit Import contains Alias, Fully qualified name and Enabled. Alias is an organizational label, not a Java `as` alias. Runtime imports use the full class name, such as `java.util.List`, after which code can use `List`.

Apply with: Workbook > Apply configured imports. Editing this table does not remove imports already applied to a live session. The current implementation saves newly created entries as enabled; turn them off in On afterward if needed.

Settings / Preferences > Spring Boot Debug REPL and MCP

Field / control	Purpose
Endpoint (optional; run configs discover their own)	Manually selected local agent endpoint. Checkbox launches discover their own automatically.
Agent JAR (empty = bundled)	Custom agent; blank uses the matching bundled version.
Agent port (0 = allocated by the OS)	nREPL agent port. Zero selects a free port; this is not the MCP port.
Connect the selected endpoint when the project opens	Try connecting to that endpoint when opening the project.
Persist REPL history (may contain application data)	Persist history; by default it remains in memory.
Show results beside source for Evaluate at Caret	Display Evaluate at Caret / Run Selection results beside source, with Inspect / Snapshot / Output actions.
AI (mcp-offline = disabled)	Built-in AI panel provider mode.
API key	Stored in PasswordSafe; IDEA's OPENAI_API_KEY environment variable is a supported fallback.
Model / Base URL	Built-in AI model and endpoint; use values appropriate for your provider.

Save with IDEA's **Apply / OK**. Agent settings do not replace the agent in an already running JVM; relaunch or attach again. Spring controls the HTTP port, these Settings control the agent port, and MCP controls the MCP port.

20. MCP: local server controls

Location: MCP, the second main tab. Connect to the running application first. The workbook's MCP action only navigates here.

Control	Function
Start MCP	Start a local HTTP server for the current REPL target, using the selected port and permissions.
Stop MCP	Stop access and close client sessions. The application keeps running.
Advanced settings...	Show or hide port, execution policy, limits, individual tools and redaction. Hiding the section preserves its settings.
Port (0 = free), under Advanced settings	Zero chooses a free port; another value requests that local port. Locked while running.
Copy client config	Copy JSON containing the URL and private Authorization token. Requires a running server.
Allow Java execution / state changes	Off by default. Main permission for execution and state changes. Reading/analysis tools also depend on their own switches and tool allowlist.
Allow HotSwap	Off by default. Separate permission for code reload; also requires the main execution switch.
Allow snapshot / CASE writes	DATA/CASE writes, import and reproduction creation; separately permitted, off by default.
Allow snapshot deletion	Separate delete permission; off by default.
Allow CASE execution	Saved CASE execution; off by default.
Share IDE recordings with MCP	Share the current IDE recording, recorded values and source. Off by default; reading does not need Java execution. See chapter 40.
Allow capture / trace changes	Permission for capture changes; off by default.
Choose allowed tools	Allow tools individually; category switches still apply.
Java / CASE mode	ROLLBACK, READ_ONLY or LIVE. MCP defaults to ROLLBACK, independently of the workbook.
Transaction manager	Exact bean name. Blank auto-selects only when exactly one manager is available.
Timeout ms / Calls per session / Result chars	100-120000 ms; 1-100000 calls; 1024-65536 characters. Defaults: 30000, 1000, 65536.
Redact likely secrets in MCP results	On by default. Redacts recognized secret patterns; not complete personal-data detection.
MCP URL	Read-only local address ending in `/mcp`. Give this to the client.
Clients / Last tool	Active client session count and last tool invoked.

The main MCP screen shows connection controls and permission categories. **Advanced settings...** opens the remaining controls listed above. The complete 82-tool catalog and all existing permission checks remain available.

Changing permissions and restarting

Stop MCP > configure switches > Start MCP. The last successful listening port is remembered for this project, including a port allocated with 0. The project token is retained in IDE PasswordSafe according to the IDE password storage settings. The copied client config remains usable after MCP or IDE restart. Choose 0 or another port to change the URL; copy the config again if the URL changes or the saved credential is cleared. Client sessions and LIVE references still close on stop; reconnect the application and start MCP explicitly.

The server listens only on `127.0.0.1`. Up to four MCP sessions can be open; they expire after 30 idle minutes. Changing or losing the REPL connection, or closing the project, stops MCP access.

What is shared?

IDE and Claude variables: separate sessions, handles and LIVE pins. **Spring beans and DATA:** shared in the same application. A separate session is not a database transaction or rollback.

Adapt the generic copied JSON to the client's format. Claude Code also requires `"type": "http"`; the next chapter gives an example.

21. Claude Code: connection and first task

Run Claude Code on the machine that can reach IDEA's local MCP server. In your application project directory, replace both placeholders and add the server:

```
claude mcp add --transport http --scope local spring-boot-repl \
  'http://127.0.0.1:PORT/mcp' \
  --header 'Authorization: Bearer GENERATED_TOKEN'
```

PORT comes from MCP URL; GENERATED_TOKEN comes from Copy client config. Include `Bearer` exactly once. This creates a personal connection scoped to the project. Check status with `/mcp` in Claude. To replace the token, remove and re-add the local entry:

```
claude mcp remove --scope local spring-boot-repl
```

Re-add it with current values and open a new Claude Code session. Do not paste the token into conversations or commit it to a repository.

File-based alternative

Preserve existing servers when editing the project's `.mcp.json`:

```
{
  "mcpServers": {
    "spring-boot-repl": {
      "type": "http",
      "url": "${SB_REPL_MCP_URL}",
      "headers": {
        "Authorization": "Bearer ${SB_REPL_MCP_TOKEN}"
      }
    }
  }
}
```

Set both environment variables in the terminal launching Claude. This is an alternative to the CLI method; configure a connection in one place. Reference: Claude Code MCP documentation.

First task for Claude: "Get the REPL status and the first 50 beans. With the context ready, check ctx.getBeanDefinitionCount() without running it, then evaluate it."

Chapter 25 is the English tool reference. The repository also provides `docs/claude-repl-instructions.md` and `docs/claude-repl-guide-hu.md` as Hungarian agent instructions and setup guidance. Connection syntax was checked against local Claude Code 2.1.76 help; a complete interactive client test has not been verified.

22. Claude Desktop and another machine

For local Claude Desktop MCP configuration, the external **mcp-remote** adapter can be used. It requires Node.js/npm. Options for pinned version 0.14.2 were checked against its published README; a full Desktop connection has not been tested.

macOS configuration: `~/Library/Application Support/Claude/claude_desktop_config.json`. Preserve existing entries and replace the placeholders:

```
{
  "mcpServers": {
    "spring-boot-repl": {
      "command": "npx",
      "args": [
        "-y", "mcp-remote@0.14.2",
        "http://127.0.0.1:PORT/mcp",
        "--allow-http", "--transport", "http-only",
        "--header", "Authorization:${SB_REPL_AUTH_HEADER}"
      ],
      "env": {
        "SB_REPL_AUTH_HEADER": "Bearer GENERATED_TOKEN"
      }
    }
  }
}
```

Restart Desktop. If npx is not found, use the full path returned by `command -v npx` as `command`. The adapter downloads on first launch. Reference: mcp-remote documentation.

Why does localhost fail in a web connector?

The claude.ai, Cowork and Desktop remote **Custom connector** connection originates in Anthropic's cloud and cannot reach your computer's `127.0.0.1`. Desktop's local MCP configuration is a separate mechanism. Use Claude Code on the same machine or Desktop's local MCP connection for this plugin. See Claude connector network requirements.

Another machine: install the plugin ZIP there, start the application and MCP there, then give its URL/token to the client running there. Transfer DATA with IDE file export/import; required DTO classes must be available on the destination. LIVE references cannot be transferred between machines.

23. Complete workflow: capture to fix

This example connects the tabs. Replace `com.example` classes and methods with your application's API.

1. Obtain real input

With an application trigger, choose **Snapshots > Capture next > Arm capture rule**, then send the request from HTTP or your usual client. Wait for **SAVED**. For a value available only as a local, use **Debugger > Capture to REPL > Resume**, then Freeze DATA.

2. Restore and inspect

Choose **Snapshots > Saved > Load**, variable `input`, then **Variables > input > Inspect**. Check fields and element types in Fields. Keep the snapshot name as the input's provenance.

3. Write a small experiment

```
// %% Service
var processor = ctx.getBean(com.example.CvProcessor.class);

// %% Experiment
var actual = processor.process(input);
actual
```

Choose **Check code**, then run the required cells separately. Use Inspect result for `actual`. Save under another DATA name to retain the starting sample.

4. Define expected behavior

Prepare expected DATA, then **Cases / Reload > Save case**. Retrieve the service through `ctx` inside the CASE; workbook variable `processor` is not transferred. **Run saved cases** establishes behavior before the fix.

5. Reload and verify

Edit Java source, then **Use open Java editor > Reload + run selected**. Treat the new run as testing modified code only after successful HotSwap. Structural changes may require restarting the app, restoring input from DATA and rerunning the CASE.

6. Give Claude the same data

Start MCP and ask: "Load DATA snapshot cv-input-42 as input, examine the relevant fields and reproduce the error using the service call we agreed on." Claude works in its own session. Request needed fields/pages instead of the whole 200 MiB object. Claude can return its result to IDEA under another DATA name.

24. Keyboard shortcuts and menus

These are the plugin's registered defaults. Keymaps and the operating system can override them. Workbook shortcuts apply inside its editor.

Action	Shortcut and scope
Run cell / selection	Cmd+Enter or Ctrl+Enter in the Workbook workbook.
Run + next	Shift+Enter in the workbook.
Complete	Ctrl+Space in the workbook.
Run Selection	Ctrl+Shift+R for selected code in a normal source editor.
Evaluate at Caret	Meta+Shift+E, or Cmd+Shift+E on macOS. Evaluate the Java expression at the cursor in the REPL.
Reload Class	Meta+Shift+R, or Cmd+Shift+R on macOS. Compile the Java class and request HotSwap.

Meta means Cmd on macOS. Evaluate at Caret and Reload Class have no separately registered Ctrl-based Windows/Linux default. Search the action in **Settings / Preferences > Keymap** and assign an available shortcut.

Code > Spring Boot Debug REPL and MCP and the Java context menu

Menu item	Action
Run Selection / Evaluate at Caret	Evaluate in the REPL session, not a method's local frame.
Advanced Editor	Show Spring Boot Debug REPL and MCP; choose Workbook for the workbook.
Sync Imports	Transfer Java source imports to the session.
Trace Method	Trace the current method.
Record Class Calls...	Record class calls with a graph and source-side input/results; see chapter 39.
Reload Class	Manually reload code.
Help (PDF) > English / Magyar	Open the selected bundled manual offline.

IDEA's **Tools** menu also contains **Attach & Inject Dev Runtime** and **Bind Spring Context**. Assign shortcuts to the 42 workbench commands; they reserve no new global default combinations. You can assign Help (PDF) a shortcut too; it opens the language picker.

Other workbench commands: search IDEA Find Action for `REPL:` to find Run, Workspace, Session and Tools actions, and assign shortcuts in Keymap. **Snapshot point...** also appears in the Java editor context menu, gutter context menu and Code > Spring Boot Debug REPL and MCP.

25. MCP tool reference for Claude

`tools/list` returns allowed tools with their exact schemas. Clients may display names with a server prefix. Use `{}` for tools without parameters; unknown keys cause errors. Required arguments are **bold** below.

Ordinary lists default to 50 rows, maximum 100. Recording tools default to 20, maximum 50; see chapter 40. `nextOffset` indicates continuation. `content` contains textual JSON and `structuredContent` an object. Check `isError` and runtime error status. For CASEs, `outcome` separately reports the test result.

Suggested sequence: repl_status > fetch needed beans/data > repl_analyze > repl_eval > inspect/save the existing handle. Analyze does not create live variables. After a context change reset; after reconnecting restore from DATA. Do not automatically repeat a business call after timeout; check effects first.

Status and reading without execution

Tool	Arguments and purpose
repl_status	None; PID, context-ready, context-epoch and capabilities.
repl_list_beans	`offset`, `limit`; bean names/types.
repl_analyze	**code**; check Java without executing it.
repl_complete	**code**, **cursor**; completion with a UTF-16 cursor offset.
repl_variables	`offset`, `limit`; this session's variables.
repl_imports	None; this session's imports.
repl_snapshot_list	`offset`, `limit`; available snapshots.
repl_snapshot_info	**name**; metadata.
repl_snapshot_versions	**name**, `offset`, `limit`; versions-json list, default 20, maximum 100.
repl_snapshot_provenance	**name**, `version`; environment and verified SHA-256.
repl_notebook_symbols	**code**; analyze declarations without executing Java.
repl_snapshot_diff	**before**, **after**, `offset`, `limit`; DATA differences.
repl_snapshot_export	**name**; small JSON preview, not a guaranteed complete export.
repl_capture_status	`rule-id`; your rule's status; omitted means the latest rule you own.
repl_capture_list	None; your rules, counters and latest save.
repl_execution_policy	None; runtime/MCP mode, managers and remaining MCP quota.
repl_execution_preflight	**code**; textual side-effect hints without execution.
repl_audit_events	None; recent runtime audit entries for your session.
repl_events	None; bounded event list.
repl_case_list	`offset`, `limit`; names, tags, disabled state and row count.
repl_case_result	**name**, `row`; your session's latest result without rerunning.

Tool	Arguments and purpose
repl_case_export_junit	**name**, package, `class`, `file`; file manifest or preview of one Java/JSON file without execution.
repl_case_load	**name**; read the definition without running it.

Java and objects

Tool	Arguments and purpose
repl_eval	**code**; Java in your persistent session.
repl_interrupt	None; request interruption of your execution.
repl_reset	None; discard session values and use the current context.
repl_bind_spring	None; bind ctx after startup.
repl_add_imports	**imports**; newline-separated Java import statements.
repl_inspect	**Exactly one:** `handle`, `var` or `event`.
repl_inspect_page	`offset`; another field page of the current object. No limit parameter.
repl_inspect_push	**revision**, **index**; open a field from your latest inspector response.
repl_inspect_back	None; return to the previous object.

Saving, triggers, cases and reload

Tool	Arguments and purpose
repl_snapshot_save	**name**, `type`, and **one of** `handle` / `var` / `event`; save DATA.
repl_snapshot_pin	**name**, and **one of** `handle` / `var` / `event`; pin LIVE.
repl_snapshot_load	**name**, `var`, `type`, `version`; default variable loadedSnapshot.
repl_snapshot_restore_version	**name**, **version**; create a new current version from earlier content without Java.
repl_workspace_export	**path**, **state-path**; write a local ZIP with existing workspace JSON. Requires execution and snapshot-write permissions.
repl_workspace_import	**path**, **prefix**; import validated contents under new snapshot names and return a mapping. Does not open the IDE workbook.
repl_snapshot_delete	**name**; delete the snapshot and all versions.
repl_snapshot_import	**name**, **json**; import a small JSON string.
repl_capture_arm	**point**, **name**, `type`, `case`, `ttl-ms`, `count`, `sample-every`. Case is an exact request-ID filter.
repl_capture_disarm	`rule-id`; cancel your waiting rule.
repl_events_start	`label`; subscribe to taps.
repl_events_stop	None; stop taps.

Tool	Arguments and purpose
repl_case_save	**name**, **input**, **expected**; `code`, `type`, `variable`, `expected-exception`, `expected-message`, `assertions-json`, `parameters-json`, `result-expression`, `imports`, `setup`, `teardown`, `tags`, `disabled`, `max-duration-ms`, `max-sql-count`, `max-sql-repetitions`, `max-hibernate-loads`, `max-hibernate-flushes`, `max-hibernate-lazy-loads`, `max-hibernate-response-lazy-loads`. Code or result-expression is required.
repl_case_run_batch	**names**; newline-separated CASE names sharing one deadline.
repl_case_run	**name**; execute a saved case.
repl_reproduction_create	**name**, **input**, `variable`, `type`, `metadata-json`; bundle from the last run without repeating it.
repl_reload	**code**; complete Java source with separate HotSwap permission. No path/file/className argument.

IDE recordings and graph (0.20 onward)

These 17 tools work with the shared IDE recording. The complete catalog contains 82 tools. Enable **Share IDE recordings with MCP**. Chapter 40 covers pagination and a Claude workflow.

Tool	Arguments and purpose
repl_recording_status	None; current IDE recording ID, view version, state and downloaded call count.
repl_recording_calls	**recording**, `view`, `query`, `errors-only`, `root`, `focus`, `thread`, `min-duration-ms`, `from-ms`, `to-ms`, `collapsed`, `offset`, `limit`; filtered call tree and call path.
repl_recording_call	**recording**, **call**, `view`; exact overload, breadcrumb, children, previous/next/nextError and source availability.
repl_recording_values	**recording**, **call**, **part**, `view`, `path`, `offset`, `limit`, `text-offset`, `text-limit`; recorded input/result/exception fields.
repl_recording_source	**recording**, **call**, `view`, `offset`, `limit`; paged captured source, filename, original SHA-256 and method line.
repl_recording_timeline	**recording**, `view` and calls filters, `offset`, `limit`; thread timing. No collapsed argument.
repl_recording_pin	**recording**, **call**, `view`; pin a downloaded call as this client's comparison reference.
repl_recording_compare	**recording**, **after**, and **one of** `before` / `reference`; optional `view`, `offset`, `limit`. Field differences and partial status.
repl_recording_select	**recording**, **call**; select the IDE node and open its source and recorded values. Requires state-change permission.
repl_recording_start	**expected**, **classes**; `sql` (string true/false, default true), `hibernate` (true/false, defaults to sql), `n-plus-one-threshold` (2-1000, default 5). Expected is the current recording ID or none; classes is 1-8 distinct exact names separated by newlines. Requires capture/trace and state-change permissions.

Tool	Arguments and purpose
repl_recording_stop	**recording**; stop that shared recording. Requires capture/trace and state-change permissions.
repl_recording_hibernate	**recording**; `view`, `root`, `call`, `event-id`, `kind`, `offset`, `limit`. Hibernate 6.6 metadata, session, entity, association, lazy/flush/cache events and related SQL IDs.
repl_recording_hibernate_findings	**recording**; `view`, `root`, `call`, `offset`, `limit`. Lazy N+1 suspects linked to SELECTs and lazy loads during response rendering.
repl_recording_hibernate_compare	**recording**, **after**, and exactly one of `before` / `reference`; `view`. Before/after Hibernate counters, differences and partial status.
repl_recording_sql	**recording**; `view`, `root`, `call` (whole subtree), `sql-id`, `offset`, `limit`, `text-offset`, `text-limit`. JDBC events and paged SQL text; parent/root, timing, datasource, source and error type.
repl_recording_findings	**recording**; `view`, `root`, `offset`, `limit`. Per-request suspected N+1 SELECT groups, counts, time and example IDs.
repl_recording_sql_compare	**recording**, **after**, and exactly one of `before` / `reference`; optional `view`. SQL statistics and differences for two call subtrees.

26. Troubleshooting from the UI

Symptom	What to check
Not connected / not READY	Application startup log, REPL checkbox and Workbook status. Bind cannot fix a Spring startup error.
No active profile	Active profiles or `--spring.profiles.active=...`, not a bare comma-separated argument.
Checkbox missing / old classloader error	Install the fresh ZIP and restart IDEA. Check supported IDE versions and any old custom agent setting.
Local variable unknown	Evaluate at Caret runs in REPL scope. Use a debugger frame or capture for locals.
Type unavailable after editing Imports	Apply configured imports; saving the table does not apply them.
Context changed	Reset, load DATA and manually run required preparation cells.
Partial preview / Partial code check	Reduce the examined data/code; navigate into a smaller object in Inspector Fields.
Capture missing	Exact point name, case ID, bridge, arming and a real matching application call. Debugger capture requires Resume.
Snapshot loading fails	DTO and Jackson availability; saved generic type. Open CASEs in Cases / Reload.
Reload rejected	Compilation error or unsupported JVM structural change. Restart the application for the latter.
HTTP environment variable missing	Play sees the IDE environment; a snippet sees the application's. Check where the variable is set.
MCP 401 / connection refused	Start MCP for the connected application; check the saved port, PasswordSafe and exact Authorization header. Re-copy config after changing the URL or clearing the saved credential.
Claude Code command error	Include `"type": "http"` in JSON configuration; URL alone is insufficient.
MCP GET /mcp fails	SSE requires the bearer token, initialized session and Accept: text/event-stream. It is not a public web page; see chapter 49.
MCP tool missing	Change permissions while MCP is stopped, then Start and update client configuration.
Variables Previous/Next do not page	Use Inspector's own Previous/Next.

Uncertain outcome: do not automatically repeat writes after a timeout or lost connection. Reset is not undo; Interrupt only requests cancellation. See chapter 28 for rollback boundaries.

27. Limits, terminology and references

Key limits

Area	Current limit / behavior
DATA snapshot	Default 200 MiB including metadata; not a heap limit.
Inline JSON	IDE: 2 MiB; MCP import: 100,000 characters, also subject to request limits.
MCP transport	Request 256 KiB; response protocol ceiling 512 KiB plus a configured 1024-65536 character limit. Large DATA stays in the runtime.
MCP session	Up to four; 30 idle minutes; 4096 distinct request IDs per session.
LIVE / events	LIVE up to 30 minutes; tap/trace 128 values for five minutes per session.
Capture rules	16 active per JVM; 1-100 saves per rule. UI TTL five minutes; runtime TTL 1 ms to 30 minutes.
Object preview	500 values, six levels, 50 children per container and 131,072 total text characters.
Live check	100,000 source characters; 200 declarations / 100,000 context characters; 200 snippets; 100 diagnostics.
Checking deadline	Five seconds, checked between compiler operations; does not forcibly stop an individual javac operation.
HTTP	10 s connection, 30 s request; 1 MiB request/response body and bounded console preview.

Always account for truncation and partial checks. A partial comparison with zero differences does not prove equality.

Terminology

Session: persistent Java evaluation workspace. **ctx:** Spring context. **Handle:** session-scoped ID for an existing result. **DATA:** serialized value. **LIVE:** live reference. **RECIPE:** saved code. **CASE:** input + expected result + executable code. **HotSwap:** applying a supported code change inside the running JVM.

Sources and updates

This guide follows the repository's `src/main/kotlin/hu/baader/repl/ui`, `ai`, `mcp`, `settings` and `runner` code, menu registration and runtime operations. Unwired legacy panels are not presented as active features.

Editable sources: `docs/repl-help-en.md` and `docs/repl-help-hu.md`. Generator `scripts/build-help-pdf.py` builds both by default. Additional Hungarian Claude instructions: `docs/claude-repl-instructions.md`; setup: `docs/claude-repl-guide-hu.md`. Version 0.22 validation: `docs/history/HIBERNATE_0_22.md`; SQL recordings: `docs/history/SQL_RECORDINGS_0_21.md`; earlier IDE compatibility: `docs/history/IDEA_2025_2_0_13_1.md`.

Help (PDF) opens the bundled copy. Install the updated plugin to update its manuals, or use these PDFs independently. UI behavior was checked against source; this does not constitute exhaustive testing of every business application or Claude client.

28. Execution modes and transactions

Below the Workbook toolbar, the application name, profiles, connection and server-confirmed execution mode are shown. This setting applies to Java and CASE execution in the IDE session. Native sessions default to LIVE. MCP uses an independent policy, defaulting to disabled Java execution and ROLLBACK mode.

Control	Usage
LIVE / DB rollback / DB read-only	Selecting a mode immediately requests the change. Display updates after server confirmation; there is no separate Apply.
Applying / Reading execution settings	A change/read is pending. Java and CASE execution are disabled for buttons and shortcuts.
Mode unconfirmed	Actual state is unknown. Session > Refresh execution settings reads it again. Failed changes also trigger readback.
Session > Execution settings	Transaction-manager bean name and timeout in one dialog. Choose a specific manager when multiple exist. Default 30000 ms; range 100-120000 ms.
Session > Refresh execution settings	Read actual mode and available managers.
Tools > Side-effect hints	Check selected or complete source for HTTP, messaging, file, process or async hints without running it.
Tools > Audit events	Write recent runtime audit entries for your session to the console.

Mode changes are disabled during execution. The tooltip shows full application name, profiles and PID when available. Profiles are informational; change Run Configuration and restart the application to activate different profiles.

ROLLBACK/READ_ONLY start a new synchronous transaction on the Java execution thread. Input restoration and CASE assertions are inside it too. The manager receives rollback after success, Java failure and returned interruption. With no manager or an ambiguous selection, execution never starts; there is no silent fallback to LIVE.

Boundary: only synchronous database work participating in the chosen manager's transaction can roll back. Nested REQUIRES_NEW, other managers, async/reactive work, HTTP, Kafka/RabbitMQ, file writes and in-memory bean mutations are outside it. Read-only is a driver/manager hint, not a universal write barrier. There is no general undo, outbound firewall or automatic mock.

Timeout requests cooperative interruption and cannot guarantee stopping code that ignores it. There is no active Java timer before transaction creation. Console and MCP responses report mode, rollback outcome and duration. Do not treat an unknown outcome as success or automatically repeat a write.

Transaction semantics follow Spring's programmatic transaction management: docs.spring.io/spring-framework/reference/data-access/transaction/programmatic.html .

29. Reproduction in one bundle

Location: Workbook > Tools: **Create reproduction**, **Export reproduction**, **Import reproduction**. Creation uses selected DATA input and the last Java execution's code/result. It cannot retrospectively reconstruct arbitrary application calls.

1. Save the failing input as DATA through Capture or Saved.
2. Load it, for example as `input`, and run a reproducing cell. The cell must be self-contained with the selected input and imports; other session variables are not bundled.
3. Click **Create reproduction**, enter a name, choose input DATA, its code variable and an optional type. Add tenant/feature flag/HTTP metadata if needed.
4. New input and expected DATA, a CASE, a RECIPE and environment DATA are created. Export a `.sbrepl-bundle` file afterward. Code is not rerun.
5. In another environment use **Import reproduction**. It imports under new names; run the CASE separately in Cases / Reload.

A checkbox in Create can include the last saved HTTP request as metadata. This does not establish that it caused the last REPL execution. The request is not sent or automatically added to the HTTP tab on import.

Environment evidence: time, Java version, PID, context epoch, thread, locale/timezone, active/default profiles and available Spring Boot/build/Git data. Original input capture context is preserved. Git/build information exists only when exposed through application properties/resources. The SecurityContext name is available only from Spring Security on the current thread.

Not restored: principal, permissions, tenant, clock, profiles, feature flags, external services or live objects. The last result is copied at Create time; a mutated object may differ from its state when the original call returned. The input DATA copy remains independent of later overwrites of its source.

The ZIP manifest contains roles, sizes and SHA-256 checksums. Import accepts only the six known files; path traversal and duplicate entries are rejected. Expanded total size is at most 200 MiB. Checksums detect corruption, not authenticate a sender or provide a digital signature.

30. Expected exceptions and capture metadata

Cases / Reload has **Expected exception** and **Expected message**. With no exception type, normal structural DATA comparison applies. When supplied, exact exception class and message are required, not subclass or regex matching. A compile error cannot count as an expected business exception.

Creating a reproduction from the last runtime-failing execution automatically makes such a CASE. This records the old failure; change expected behavior after fixing it, otherwise the test still demands that failure. Expected DATA remains required; an exception bundle stores its error description there.

Capture tenant and feature flag information on the application thread using the bridge overload:

```
SnapshotHelper.captureLazy(
    "order-processing", requestId, () -> inputDto,
    java.util.Map.of("tenant", tenantId,
                     "feature.newFlow", "true"));
```

Metadata allows 32 text fields, keys up to 128 and values up to 2048 characters. Recognized secret-named fields are redacted. These are evidence, not automatically restored thread-local values.

For repeated capture choose **Capture count = 5**, name `order-${sequence}`, an optional exact case ID, then **Arm capture rule**. Your rule table shows counters and the latest snapshot. Rules cannot have overlapping planned snapshot names. An existing inactive snapshot gets a new version on a same-name save. View persistent versions in Snapshots > Versions.

31. Audit, secrets and future work

Runtime and MCP write private local JSONL logs under **~/.java-repl-audit**, with separate files for runtime and IDE PIDs. Files rotate above 4 MiB, retaining the current file and five earlier copies. POSIX permissions are 700 for the directory and 600 for files.

Entries include session/client, operation, start/end, duration, code and code hash, plus runtime context epoch. Capture saves are logged separately. CASE logs contain the source actually loaded. An operation cannot start without a start audit entry; a failed completion audit reports that effects may already have happened. Audit events provides a bounded preview of up to 100 entries for your session in the current file.

Recognized password, token, Authorization, cookie and similar patterns are redacted from audit, history and, by default, MCP responses. If the AI panel detects such a pattern before sending, it updates the prompt preview and requires another review and Send click. Detection is heuristic and does not find every secret or personal datum. Original workbooks, DATA/RECIPE content, transcripts and bundle business data are not automatically anonymized. Review exports.

MCP permissions control tools; arbitrary Java execution is not a sandbox. Allowed Java runs in the application's JVM with access to its classes and beans. There is no reliable package, bean or method allowlist for arbitrary code. Quotas restart with a new session; logs are neither tamper-proof nor retained indefinitely.

Delivered in 0.14: transactional execution, side-effect hints, reproduction creation/export/import, finer MCP permissions, audit, multiple capture rules and expected-exception CASEs.

Delivered in 0.15: field assertions, parameterized CASEs, setup/cleanup, JUnit export and corresponding MCP tools; covered next.

Available in later chapters: notebook state, workspace saving, snapshot versions, source snapshot points, call graphs/timeline, SQL/N+1 and Hibernate observation; see chapters 35-42.

Possible further work: automatic snapshot schema migration, outbound blocking/stubbing, general reactive tracing, remote container/Kubernetes connections, a standalone headless/CI runner, encryption and a team registry. These are not claimed as implemented features.

32. CASE 2.0: fields and assertions

Existing CASEs still run in **Cases / Reload**. **Load selected** opens a saved definition; **Save case** saves fields without running code. **Run saved cases** executes the saved version, so save edits first.

Field / subtab	Purpose
Code	Java statements. Legacy CASEs use the last JShell value as the result.
Result expression	Optional separate expression such as input.size(), evaluated after Code. Required for JUnit export; omit a trailing semicolon.
Assertions	JSON settings; blank or {} compares the complete expected DATA.
Parameters	Multiple saved input/expected pairs for the same code. Blank uses the top DATA selectors.
Setup / cleanup	Separate fields for imports, setup Java and cleanup Java.
Options	Comma-separated tags, Disabled and Maximum code duration in milliseconds.
Export JUnit ZIP	Generate a Java test and JSON resources from one selected saved CASE without running it.

Assertions use JSON Pointer paths: empty string means root, **/items/0/name** means the first item's name. **/items/*/id** selects each item's id at one level. Escape / in a key as ~1 and ~ as ~0. The * is reserved as a wildcard; this is not full JSONPath.

```
{
  "ignore": ["/id", "/items/*/createdAt"],
  "unordered": ["/items"],
  "numericTolerance": {"/total": 0.01},
  "timeToleranceMs": {"/updatedAt": 1000}
}
```

include: compare only the specified subtrees. **ignore:** skip specified subtrees. **unordered:** compare list elements independent of order using one-to-one matching; duplicates matter. Supports up to 200 items. Tolerance matching is not greedy, avoiding false differences with overlapping ranges.

numericTolerance: absolute nonnegative numeric difference; strings are not coerced to numbers. **timeToleranceMs:** difference between ISO instant or offset date/time strings. LocalDateTime without an offset is unsupported. An exact path overrides a wildcard; the first matching wildcard takes precedence among wildcards.

```
{
  "compareSnapshot": false,
  "checks": [
    {"path":"/items", "op":"hasSize", "value":2},
    {"path":"/items/*/code", "op":"matches",
     "value":"[A-Z][0-9]+", "match":"all"},
    {"path":"/items/*/active", "op":"equals",
     "value":true, "match":"any"}
  ]
}
```

Check operators: **equals**, **contains** (substring or list item), **hasSize** (list/map/string), **matches** (full regex match), **exists**, **notNull**, **isNull**. Match defaults to all; any requires one match. Missing paths or empty wildcard results fail. Ignore does not disable an explicit check. compareSnapshot=false requires at least one check; still select expected DATA.

Limits: 100 paths/checks, 64 KiB settings JSON and 200,000 comparison steps. Depth, regex or work limits cannot produce false PASSED results: they produce INCONCLUSIVE, or FAILED when a difference is already proven. Unknown options and invalid JSON are rejected when saving.

33. Parameters, lifecycle and results

Parameters accepts a JSON array with 1-20 distinct IDs. Every row refers to existing DATA names:

```
[
  {"id":"small", "input":"input-small",
   "expected":"expected-small"},
  {"id":"large", "input":"input-large",
   "expected":"expected-large"}
]
```

Execution order: fresh session and transaction boundary, input load, imports, setup, Code, Result expression, assertions, cleanup, then rollback according to the chosen mode. Each next row starts with a new input instance and transaction. Workbook variables are not transferred; Spring beans and LIVE side effects are shared.

Maximum code duration is an after-the-fact assertion of 1-120000 ms. In the runtime it measures Code and result expression evaluation, including JShell compilation; input loading, setup, JSON comparison and cleanup are excluded. It is not an interruption timer. The execution policy's separate cooperative deadline still applies and is shared by all rows of a CASE.

A **Disabled** CASE produces SKIPPED without running code. Tags are stored, listed through MCP and exported as JUnit @Tag annotations. Folders and tag-based execution filtering are not implemented.

Setup failures cannot satisfy expected-exception assertions. **Cleanup** is attempted after normal results, assertion failures and execution failures, but skipped after interruption. A cleanup failure makes the CASE ERROR. Transaction closure remains the execution policy's responsibility. Application code that ignores interruption may not stop immediately.

Rows run sequentially. FAILED permits subsequent rows; ERROR and CANCELLED stop the remaining rows. Reports include aggregate and per-row status, duration, differences, output and rollback details. The detailed report is a bounded preview. Inspect actual works directly for one row; for multiple rows choose the relevant CASE event in Events.

MCP: **repl_case_list**, **repl_case_run_batch**, **repl_case_result**. Batch names accepts 1-20 newline-separated CASE names, up to 100 parameter rows in total. All definitions and DATA references are validated before execution. Separate CASE permission is required; the client cannot override MCP's mode or deadline.

repl_case_result reads the latest result for your session without executing. Row is a zero-based index. Up to 20 CASE results are retained and expire after reset/context change. Do not automatically repeat batch or CASE calls after timeout.

34. Export a CASE as JUnit 5

1. Choose input and expected DATA in Cases / Reload. Export requires a full Java type in **Optional input type**, for example java.util.List<java.lang.Integer>.
2. Put Java statements in Code and the final value in Result expression. Example Code: input.add(1); and Result expression: input.size(). Put imports in the separate Imports field.
3. Save, select the CASE and click **Export JUnit ZIP**. Enter a Java package, test class name, execution mode and transaction manager if needed. Export defaults to ROLLBACK.
4. Copy the ZIP's src/test/java and src/test/resources contents into the application's test directories. Review JSON data, test profiles and external dependencies.
5. Run the application's usual build, for example ./mvnw -Dtest=ReproductionTest test or ./gradlew test --tests 'reproduction.ReproductionTest'.

Contents: a @SpringBootTest + @ParameterizedTest class, ApplicationContext injection named ctx, input/expected JSON per row, assertions.json, a portable assertion helper sharing runtime source, and README. AssertJ checks the results. Requires Java 17+, spring-boot-starter-test, Jackson and spring-tx. Normal Maven/Gradle tests provide JUnit XML; IDEA and the REPL agent are not required.

Export checks Java syntax only. The application's build checks types and test dependencies. Convert JShell-only declarations, agent helpers and earlier session aliases into ordinary Java first. Customize the generated mapper for application-specific Jackson codecs/mixins. Runtime profiles and secrets are not automatically activated in the test.

Generated tests use separate transactions and rollback per row according to the selected mode; multiple managers require an explicit name. Cleanup can access input and ctx, not exercise-method locals. JUnit duration assertions measure code without JShell compilation, so timing is not exactly equivalent. Interruption occurs on the original execution thread to keep code and transaction together.

Total export size is limited to 16 MiB; reduce large snapshots to smaller reproduction fixtures. Legacy .sbrepl-bundle v1 carries one fixed input/expected pair; use JUnit ZIP for parameterized or DATA-rebound CASEs. Chapters 35-37 cover notebook/workspace saving and snapshot versions. A separate bundle CLI remains planned.

Claude: repl_case_export_junit(name, package, class) returns a file manifest. Add file to retrieve one source/JSON preview without writing files or executing code. Each inline file is limited to 64 KiB, further constrained by MCP result limits and redaction. Do not construct test fixtures from truncated or redacted output; use IDE ZIP export for the complete package.

See JUnit 5 documentation for parameterization/timeouts. See Spring test transactions and chapter 28 for transaction boundaries.

35. Notebook: cell state and dependencies

Location: Workbook workbook and Cells result tab; sequence execution is in Run. Existing `.jsh` files and `// %%` boundaries remain supported.

Results at the cell: each cell shows execution number, status, duration and short output underneath. Stale indicators name the changed input cell when known. Click the run marker in the left gutter for cell actions.

Cell action	Effect
Run	Execute the entire cell, clearing an earlier text selection.
Inspect	Open that run's live result without rerunning the expression.
Snapshot	Save DATA from the existing result. One dialog accepts name, existing destination and optional Java type. An existing name creates a new version.
Output	Open the cell's stored output in Cells.

In a narrow editor, Run and Output remain visible; right-click for other actions. Reset/reconnect disables Inspect/Snapshot for earlier live results. After source edits, an explicitly marked earlier result remains inspectable while its reference is valid. Only saved DATA freezes later object mutations.

CIDER-style source results: Evaluate at Caret and Run Selection leave results beside code with Inspect / Snapshot / Output links. Output opens an expandable structured view. See chapter 19 for the setting. Execution still uses REPL scope; transfer method locals through the debugger or capture a snapshot point as described in chapter 38.

Indicator / field	Meaning
Run [12]	The cell's latest execution number; each actual cell execution gets a new number.
SUCCESS / ERROR / INTERRUPTED / NEVER	Success, error, interruption or never executed. Success can coexist with a stale warning.
modified	Current source differs from the source associated with the stored result.
stale	Inputs, session, cell order or external state changes may have invalidated the result.
Last run / ms	Start time and duration of the latest execution.
Depends on	Cell dependencies. The lower view shows declared names, possible inputs and the cell's own output.
Go to cell	Move the cursor to the selected row's source.

Button	Behavior
Analyze dependencies	Analyze declarations with JShell without execution or installing them into the live session.

Button	Behavior
Run above	Run nonempty cells before the current cell in source order.
Run from here	Run from the current cell to the workbook's end.
Run affected	Reanalyze dependencies, then run this cell and its known dependants in source order.
Restart + run all	Confirm, reset the session and run all nonempty cells sequentially.
Run all	Run cells sequentially without resetting.

Sequences stop at the first error, edit, connection change or Interrupt. Effects of the active cell are not undone. A text selection is a separate snippet and does not mark the whole cell as executed. External editor evaluation, variable loading and HotSwap can mark earlier cell evidence stale.

Dependency limits: analysis uses declarations and possible Java identifier references. Method calls, statements, arrays and unanalyzed cells conservatively include earlier cells as dependencies. Alias, reflection and external bean-state tracking are incomplete. No automatic topological reordering or cycle resolution occurs. A dependency indicator does not prove an operation can be safely repeated.

Automatic checkpoint: workbook, recent cell results and execution order are saved per project inside IDEA's system directory. Limits: 200 cells, 16,384 output characters per cell, 1000 execution entries and 8 MiB workspace metadata. Checkpoints may contain source and application data but are not stored in VCS. Reopening shows earlier results as stale; Java objects have not been resurrected.

36. Saving and opening a workspace

Location: Workbook > Workspace; Save workspace also has a primary icon. A `.sbrepl-workspace` is a checksummed ZIP, not an executable package. **Open workbook** still opens only `.jsh`/text.

Button	Contents or action
Save workspace	Workbook, cell IDs, results, execution order, imports, DATA provenance bindings, Inspector bookmarks, optional HTTP requests and transcript. When connected, also bundles current persistent DATA / CASE / RECIPE files and older versions needed by DATA bindings.
Open workspace	Validate, then replace the editor workbook after confirmation. When connected, import snapshots under a new prefix and rewrite CASE input/expected and parameter references. HTTP requests receive new IDs.
Workspace info	Show saved environment, DATA bindings, bookmarks and variables without DATA provenance.
Restore DATA binding	Explicitly select a saved DATA version to materialize into a Java variable. May run DTO constructors and replace an existing variable.
Insert saved imports	Insert saved imports into the editor; Run separately to apply them.

Workflow: save needed values as DATA, load them into named variables, then Save workspace. In another session, Open workspace > Workspace info > Restore DATA binding; run only necessary preparation and experiment cells. Snapshot names embedded in arbitrary source/RECIPE code cannot be rewritten generally; adjust them to imported names if needed.

A DATA binding points to frozen provenance. A live object may have changed since loading; the bundle does not claim every variable exactly matches its snapshot at export time. Other Java objects, LIVE pins, handles, bean state, databases and external services are not saved or restored. Profiles, principal, tenant, timezone and versions are informational; import does not change the environment.

Inspector bookmarks: after opening from a named variable, Bookmark saves the current path. Open bookmark resolves it in the current session; restore its root variable first. Resolution stops for missing, ambiguous or unverifiable paths above 10,000 children. Remove bookmark deletes the selection. For a result/handle root, Bind current first, then open that variable from Variables.

Limits: 200 MiB total uncompressed data and a ZIP of at most 200 MiB; 8 MiB metadata; up to 1200 snapshot entries. Metadata will not fit alongside a full 200 MiB snapshot; use separate snapshot export or a smaller projection. Import prepares and validates all objects before writing collision-free new names, leaving existing snapshots untouched. This is not a crash-atomic multi-file database transaction.

Offline: Save workspace stores the local workspace without application snapshots. Open workspace loads source offline; reconnect and reopen the bundle to import DATA. Full export/import requires the same filesystem for IDEA and the JVM. Recognized secrets in HTTP requests and optional transcripts are redacted; source, cell output and DATA remain faithful copies and need review before export. Bundles are not encrypted.

37. Snapshot versions and provenance

Location: Snapshots > Saved. Select a persistent snapshot and click **Versions**. LIVE has no on-disk version history.

Button / column	Function
SHA-256 / Current	Full envelope content hash and whether it is current.
Recorded / Kind / Bytes	Record time, DATA / CASE / RECIPE kind and file size.
Provenance	Verified hash, original capture time, Java/Spring environment, active profiles, context epoch and available Git/build data.
Load version	Load older DATA into a new variable without changing the current snapshot.
Restore as latest	Create a new current version from older content with restoredFrom. Current and historical files remain. Does not execute Java.
Refresh	Reload history. The list is a preview; complete SHA verification occurs when opening/restoring.

New saves use private content-addressed files. The earlier v1 current-file format remains; the first new save also archives the old value. Each name can have up to 100 versions. There is no automatic history deletion: at the limit, saving fails and old data remains. Choose a new name or deliberately delete the complete old snapshot after exporting it. **Saved > Delete** removes the current file and all versions.

Provenance refers to original capture. Workspace import adds sourceApplicationId/importedFrom; restore adds restoredFrom/recordedAt. The Java field-schema SHA-256 describes reflected fields of the root type, not a complete Jackson schema or DTO migration. A checksum detects corruption but does not replace a signature or encryption. Automatic schema migration, restoring old DTO classes and a full-history team registry are not implemented.

Claude tools: **repl_snapshot_versions**, **repl_snapshot_provenance**, **repl_snapshot_restore_version**, **repl_notebook_symbols**, **repl_workspace_export**, **repl_workspace_import**. See chapter 25. Workspace export's state-path points to existing workspace JSON on the JVM machine; MCP does not collect the IDEA editor for you. Import returns name/version mappings without replacing the native UI workbook.

38. Snapshot points in Java source

Goal: save a local variable as DATA before a selected source line executes on a subsequent application call, without adding a helper call to application code. This uses IDEA's Java debugger; **Debug + Enable Spring Boot Debug REPL and MCP** is required.

1. Start the normal Spring Boot configuration in Debug with the REPL checkbox enabled.
2. Choose an executable Java line before which the target variable is already initialized, for example the line after creating `order`.
3. Select expression `order`, then right-click > Spring Boot Debug REPL and MCP > **Snapshot point…**. The gutter context menu also offers it; verify the expression there.
4. Enter a snapshot name such as `order-input`. **Java expression** is evaluated in that line's local frame. Java type is optional; provide the full generic type for a list if needed. **Capture attempts** defaults to 1, maximum 100.
5. Save to place a purple snapshot marker in the gutter, then trigger the application request.
6. Before the line executes, the debugger records the value and resumes the application. The Debug console reports success/failure. Open **Tools > Snapshots > Saved**, click **Refresh** if it was already open, and load the snapshot into the REPL.

The purple dot marks a configured point; placing or clicking it does not save a value immediately. A later application call must reach that line. After upgrading, restart IDEA and the Debug application. The 0.25.0 release connects this custom point to the Java debugger, fixing earlier markers that could appear without capturing.

Managing the point

Use **Configure snapshot / rearm…** in the marker's context menu, or invoke Snapshot point again on the same line. **Save and rearm** resets the counter. **Remove point** removes the point while preserving saved DATA. The native breakpoint switch temporarily disables it.

Points persist in the project's breakpoint settings and follow source-line movement. A new JVM launch starts a fresh counter. A project supports up to 64 points. `.sbrepl-workspace` does not export debugger breakpoints. If a normal breakpoint occupies that line, choose another line or remove it first.

Why was no snapshot saved?

Normal Run mode cannot detect debugger line hits. Uninitialized variables or invalid expressions cannot be evaluated. A serialization failure also consumes an attempt; fix it and rearm. Such failures do not replace existing DATA. Read details in the Debug console.

Capture runs on the hit thread. The debugger briefly suspends it for evaluation/serialization and resumes without manual Resume on success. Large objects can slow requests; the default 200 MiB limit still applies. Prefer a simple variable/field since method calls and custom serializers may have side effects. This does not guarantee an atomic image of an object mutated by another thread.

Claude/MCP: use existing snapshot listing, info, loading and Inspector tools for the resulting DATA. No new MCP tool creates these points or controls the Java debugger.

39. Recorded call trees and source-side values

Goal: see which methods ran with which inputs and results in selected Java classes. Selecting an earlier graph node opens its class source with recorded values. **Run + Enable Spring Boot Debug REPL and MCP** is sufficient; no debugger is required.

Make a first recording

1. Launch Spring Boot with the REPL checkbox and wait for the connection.
2. Open the Java class. Right-click > Spring Boot Debug REPL and MCP > **Record Class Calls…**, also available from Code and Find Action.
3. The dialog fills the full class name. Add more names, one per line, up to eight classes total. Only declared concrete methods in those classes are recorded.
4. Start recording, then use the application, for example by sending the HTTP request under investigation. Recording observes real calls; it does not trigger the business operation for you.
5. Open **Tap / Trace > Recorded calls**. Nodes show call number, class/method, success/error, duration, a short result and thread.
6. Click a node. A **CALL #…** block appears beside the method. **Input / result** opens a resizable viewer; **Call graph** returns to the recording. The graph also supports up/down arrows and Enter.
7. **Stop recording** stops new calls; completions of calls already being recorded may still arrive. **Save recording…** preserves it for later review.

Selecting both processor and parser classes can show `parse` nested under `process`. Selecting only the processor does not automatically include parser internals. Repeated/recursive calls are separate nodes. Actual parameter types identify overloads.

Buttons and views

Control	Purpose
New recording...	Choose classes for a new recording. Save the earlier one first if you need it.
Stop recording	Stop recording while the application continues.
Root	Show one entry call and its descendants, or all entry calls.
Follow latest	Follow newly downloaded calls. Manual node selection turns it off so you can examine an earlier call.
Values in source	Show newly recorded results in open source. Manual selection can open a call's values separately.
Clear source values	Remove source-side blocks, retaining the recording. Disable Values in source to prevent new automatic blocks.
Open source	Reopen selected source with its recorded input and result.
Input at entry	Arguments captured at method entry; unnamed parameters appear as `arg0`, `arg1`, etc.
Result at exit	Captured return value; void and exceptional return are distinguished.
Exception	Exception type and capturable message. Recording does not call custom overridden message-producing code.
Tree / Formatted / Raw	Expandable field tree, formatted and raw views. Expand / Collapse affects the recorded tree.
Inspect live	Inspect the still-available live event object. It may have changed; the recorded view shows earlier values.
Save recording...	Save source, graph and recorded values as `.sbrepl-recording`.
Open recording...	Open a saved recording, even without an application connection.

In a narrow window, graph and details stack vertically. Ordinary edges connect observed calls on the same thread. With async capture enabled, explicit executor handoffs link worker calls to their submitter (chapter 47); unsupported work keeps separate roots.

Graph navigation and filtering (0.19)

Call graph cards show method name, status, duration, named input and captured result. Status text has a separate success/error/running/incomplete marker. **PARTIAL PREVIEW**, **VALUES UNAVAILABLE** and **INCOMPLETE** distinguish partial, unavailable and unfinished evidence. Hover for the full signature, parameter names and a larger preview. Expand the full recorded field tree in **Values**.

Control	Purpose
+ / minus, percentage	Zoom. Ctrl or Cmd + wheel keeps the point beneath the cursor in place. Plain wheel scrolls vertically; Shift + wheel horizontally.
Drag	Pan with the left or middle mouse button. Releasing a drag does not accidentally select a node.

Control	Purpose
Fit graph	Fit visible cards in the viewport. Text may shrink on a large tree; Show selected restores a readable view of the selected call.
Show selected	Reveal the selected call and expand its ancestors. Clears filters if they hide it.
+ / minus on a node	Expand/collapse a branch. A collapsed node reports hidden calls and errors.
Collapse all / Expand all	Collapse/expand branches. Active filters keep paths to matches open.
Search	Case-insensitive substring search in classes, methods, parameter names and recorded value text. Cannot search unrecorded or not-yet-downloaded values.
Thread / Errors only / Min ms	Combine thread, error and minimum recorded-duration filters.
Focus branch	Show the selected subtree and its callers.
Clear filters	Clear query, root, thread, error, duration, time-range and branch filters.
Detach window…	Open this browser in a separate resizable window. Closing it or choosing Return to tool window returns it to the panel, retaining the recording.

Filtering preserves the caller path. Retained callers have dashed borders and **caller path** labels. Match counts exclude these context-only nodes. Search, filters and manual zoom/scroll/pan turn off Follow latest, preserving the chosen view during refreshes. Re-enable it with the switch. Opening another recording starts a new view.

Move between recorded calls

Previous, **Next** and **Next error** follow calls matching current filters, including matches inside collapsed branches. **Caller** opens the immediate recorded caller. Ordering uses IDs assigned at entry; it does not wrap after the last call.

The clickable breadcrumb might read `#1 Controller.handle > #2 Service.process > #3 Parser.parse`. Each element opens that call's source and values. Graph, Values, Compare calls and source blocks share the selection.

Browser shortcut	Action
Up / down in graph	Previous / next visible card.
Left / right in graph	Collapse, move to caller or expand.
Enter in graph	Open selected source.
= / minus in graph	Zoom in / out.
Home in graph	Fit graph.
Alt + left / right	Previous / Next within the browser.
Alt + down / up	Next error / Caller within the browser.

These keys apply when the graph/browser has focus and do not change source-editor or REPL shortcuts. Navigation visits recorded calls without executing business code.

Pin a reference and compare fields

Select a completed, downloaded call and click **Pin reference**, then select another node. **Compare calls** shows differences in recorded input, result, exception and status. The reference remains fixed as selection changes. **Repin reference** replaces it; **Clear reference** removes it.

The table shows field path, change, reference and selected values. Selecting a row opens larger text below. **ADDED**, **REMOVED**, **CHANGED** and **UNKNOWN** are also color-coded. JSON strings are compared structurally; arrays retain index/order semantics. Repeated JSON field names remain distinct occurrences.

Partial, missing or cyclic-reference previews produce explicit uncertainty. Unrecorded data is not assumed added or removed. Identical displayed fields do not prove complete live-object equality. At most 2000 difference rows and 8192 characters per field value are displayed; further differences mark the comparison incomplete.

Different methods/overloads can be compared with a header warning. Both durations are shown as informational, instrumented timings. Opening a new recording clears the local Java reference selection.

Timeline and time-range selection

Timeline shows time-scaled call bars per thread. Nested and overlapping calls use separate lanes. Clicking a bar opens source and recorded data like a graph node.

Drag horizontally to select a time range. The graph retains overlapping calls and their callers; other timeline calls remain dimmed for context. **Clear time range** removes only that filter; **Clear filters** removes all filters.

The axis uses milliseconds since the earliest recorded start. Start times use the millisecond system clock; duration is measured separately as elapsed time. Clock adjustments and recording overhead affect the display. Running/incomplete calls have no complete end time; a short marker shows only recorded evidence. Supported cross-thread tasks are linked only with async capture enabled (chapter 47).

What is preserved?

Input display data is copied at entry, result data at exit. Later list mutations do not change earlier recorded values. The browser expands already captured fields. Opening a recording does not rerun methods, restore JVM stacks or replace typed DATA snapshots.

At recording start the plugin also saves available Java source. If edited afterward, selecting a node opens the captured version in a **read-only** editor without replacing the working file. Captured source does not prove it matches running bytecode: compile and reload changed classes before recording. Start a new recording after HotSwap.

When source or an overload cannot be resolved, the panel explains it and values remain inspectable. Existing source blocks are marked as belonging to earlier code after edits.

Storage: one current recording per session, up to **200 calls** and **32 MiB display data**. Reaching a limit stops further recording while retaining earlier parents. The dropped indication means a limit was hit; it is not a complete count of every later application call. Limits also include 64 nested observed call levels, 64 concrete method names per class and 16 instrumented classes across all sessions.

Value preview: up to 500 nodes per tree, six levels, 50 children per node and the first 32 arguments. Total text budget is 131072 characters; a deeper string is limited to 4096. Limited portions are marked. Inspect the live object while available or save targeted DATA; its 200 MiB limit is separate.

Default display recording reads fields without invoking application getters or custom serializers. Separately enabled Capture replay DATA uses snapshot serialization (chapter 43). Work occurs on the calling thread and can slow the call; this is not a precise performance profiler. Objects mutated by other threads are not captured atomically. Start with a few focused classes.

Offline file: up to 64 MiB; source up to one million characters per file and four million in total. Files can contain application data and source without encryption. Opening validates format, parent links, sizes and source checksums, detecting corruption without authenticating origin. Calls still running when saved, or unfinished after disconnect, become **INCOMPLETE**. After connection loss only downloaded evidence can be retained. Save recordings separately; workspace export does not include them.

Line-local values: recordings show method inputs, results and call order. Use a chapter 38 snapshot point in Debug mode for a local at an arbitrary internal source line.

Claude/MCP: 0.20 introduced 11 tools sharing the current graph, values, source and comparisons. Later SQL/Hibernate additions bring recording tools to 17; the next chapter explains their shared behavior.

40. AI-agent access to call recordings

All recording tools require **MCP > Share IDE recordings with MCP**, off by default. Reading does not require Java execution. **Choose allowed tools** can restrict access further, for example source sharing. Pin changes only that client's frozen reference; select changes IDEA selection and requires the main state-change permission. Start/stop also require capture/trace permission.

Chapter **25** lists arguments for all 17 tools. This chapter covers shared permissions, reading order and comparison.

- Start with `repl_recording_status`. When `available=false`, nothing is shareable; start with `expected="none"`. Otherwise use the current `recording` ID. Stop an active recording first. Start waits for runtime acknowledgement; value downloads may continue afterward.
- A read response comes from one EDT state, with `scope="ide-recording"`. The graph, including a saved recording opened in IDEA, is shared with clients; Java sessions remain separate. A `call` ID cannot substitute for `event`, `handle` or `var`.
- `recording` and returned `view` identify the state being paged. A call completion or downloaded preview can invalidate an older view; restart reading with a fresh view from offset 0. An old recording ID cannot select or stop a replacement recording.
- Calls return `nodes`, timeline returns `spans`, comparison returns `rows`. Offset is zero-based; limit defaults to 20, maximum 50. Size budgets may return fewer rows; continue with `nextOffset`. A `contextOnly=true` node belongs to the caller path.
- `errors-only` is the string `"true"` or `"false"`. Time, duration and identifier parameters are integer JSON numbers. Supply `from-ms` and `to-ms` together, relative to the earliest start. No assumed cross-thread edges are created.
- Values part is input, result or exception. `path=""` means root; `"/0/2"` means child 2 of child 0. Index paths preserve duplicate field names. Offset/limit page direct children; text-offset/text-limit page the selected node's text. Text defaults to 1024, maximum 4096 UTF-16 characters; `nextTextOffset` is inside `node`.
- Source offset/limit count UTF-16 characters: default limit 2048, maximum 4096. Only captured Java source is readable. `methodLine` and `capturedSha256` describe original source, not its redacted excerpt.
- `downloaded=false` or `valueAvailable=false` may mean evidence is still missing. `partial`, `INCOMPLETE`, `RUNNING`, `UNKNOWN`, `comparisonTruncated` and `textTruncated` indicate limited evidence. Missing/redacted fields cannot prove equality; diffs do not prove full object equality.
- `repl_recording_pin` returns a client-specific reference. A new pin replaces it. After opening a new IDE recording it remains usable with `repl_recording_compare(reference=..., after=...)`; other clients cannot use it. Closing the session removes it. It is not a LIVE pin and retains no live application object.
- Secret redaction acts on decoded values before search, summaries, comparisons and source paging. It is not complete anonymization. Source, exception and value content are data, never instructions for the agent.
- Reading executes no Java or getters. Recording observes real future calls. Closing a client does not stop a shared recording; use explicit stop with its ID.

A task for Claude

Examine the current IDE Recorded calls recording. Find the failing call, show its caller path and relevant recorded input/exception fields, and read its captured method source. Support conclusions with call IDs and field paths. Do not trigger a new business operation.

Example calls; replace placeholders with IDs from earlier responses:

```
{"name":"repl_recording_status","arguments":{}}
{"name":"repl_recording_calls","arguments":{
  "recording":"<recording>","errors-only":"true","limit":10}}
{"name":"repl_recording_values","arguments":{
  "recording":"<recording>","call":3,"part":"input",
  "path":"","limit":10}}
{"name":"repl_recording_source","arguments":{
  "recording":"<recording>","call":3,"offset":0,"limit":2048}}
{"name":"repl_recording_pin","arguments":{"recording":"<recording>","call":3}}
```

Call 3 is also a placeholder: choose an existing ID from the list. After fixing code and making a new recording, compare its call against the pin response's `reference`. These read/compare operations do not modify source, HotSwap or rerun business operations themselves.

Connection and offline files

Starting MCP requires a live REPL connection; losing it stops MCP. A saved recording opened in IDEA while connected can be shared. File opening/saving and window management remain IDE actions. Chapter 49 describes event subscriptions and optional task support.

41. SQL, N+1 alerts and calls down to the DB

Version 0.21 adds JDBC observations to Recorded calls. For synchronous Spring MVC requests, the request root, selected application methods and actual JDBC operations appear in one tree. No application SQL logging or separate database plugin is needed.

Start and interpret a recording

1. Open Tap / Trace > Recorded calls > **New recording...** and select important controller/service/repository classes (1-8).
2. Keep **Record JDBC/SQL and synchronous Spring MVC requests** enabled. **Suspected N+1 repetition threshold** defaults to 5, range 2-1000.
3. Trigger the HTTP request or entry method in the application. Recording itself does not send a business request.
4. **Stop recording**, then select the request under **Root**. Concurrent requests and separate entry calls have separate SQL roots.
5. Open **SQL & N+1**. Expand groups; each execution has an SQL ID and its real parent call.

A JDBC node shows normalized SQL, execution count, client-side duration and datasource class/instance identity. Connection acquisition is measured separately and excluded from SQL execution totals. Total Java-call time includes JDBC, other application work and observation overhead.

No nodes are invented for unselected Java methods. Chapter 47 adds opt-in Executor/@Async/CompletableFuture handoffs; Reactor, R2DBC, database execution plans and locks are not tracked. JDBC timing is client-side; synchronous observation adds overhead.

Controls and views

Control	Behavior
Group SQL	Group graph operations with the same parent, SQL template, datasource and call site. Disable to see individual executions.
SQL & N+1	Details: count, duration, datasource, source class/method/line, thread, root and error type.
SQL / datasource / source + Filter	Search recorded SQL templates and metadata; Enter also applies the filter.
Suspected N+1 only	Show repeated SELECT groups reaching the threshold.
Open originating call and source	Select the SQL event's recorded application caller and open source/values.
Pin SQL baseline	Preserve SQL data for the current Root filter. Survives new recordings and opening offline files.
Clear baseline	Remove the SQL reference. Java-value Pin reference is separate.
Timeline	Show each JDBC operation's actual interval; graph grouping does not create an artificial continuous span.
Save / Open recording	Save SQL evidence and the configured threshold. Older files without SQL remain readable.

What does suspected N+1 mean?

Within one request/root, detection looks for repeated normalized SELECTs sharing a datasource and call site. By default, five or more executions produce **suspected N+1**. Ancestor nodes also receive warnings and SQL counts.

Example: Hibernate loads five orders, then each order's customer separately: 1+5 SELECTs. The integration test's fetch-join version returns the same result with one SELECT. Choose a fix appropriate to the actual query: fetch join, entity graph, batch fetching or a focused DTO query.

Repetition alone does not prove a bug; intentional repeated reads and polling also repeat SQL. This is a diagnostic threshold, not an automatic code change. Fetch joins with pagination or multiple collections can introduce other problems; verify result correctness as well as query count.

Compare before and after a fix

Select the failing request's Root and click **Pin SQL baseline**. Fix and compile code, using HotSwap for supported changes. Start a new recording, trigger the same request and select its Root.

Comparison shows differences in SQL count, total JDBC duration and maximum repetition. Keep datasource, input, cache and environment comparable. One faster run is not performance proof. The panel explicitly marks partial recordings.

CASE limits and JUnit

Cases / Reload > Options includes **Maximum SQL executions** and **Maximum SQL repetition**. Blank means no SQL assertion. Zero is valid, for example to check code that must not access the database. Range: 0-1000000.

The first counts JDBC execute/executeQuery/executeUpdate and batch executions, including failed SQL. One executeBatch counts as one execution; batch item count is not claimed as query count. The second groups by root, datasource and SQL template, including operations other than SELECT. Connection acquisition counts toward neither.

Measurement covers Code and Result expression only. Input loading, Imports, Setup, Cleanup and JSON comparison are excluded. Exceeding a threshold is **FAILED**. Incomplete SQL evidence cannot pass an upper-bound assertion: the result is **INCONCLUSIVE** or **ERROR**. Without the agent, runtime SQL checks stop with an error.

JUnit ZIP preserves SQL limits and includes a portable DataSource proxy and SQL normalizer. The generated JUnit/Spring test runs without an agent. JDBC must pass through Spring-managed DataSource beans. Direct DriverManager, native unwrap, async work or injection of concrete pool classes requires adapting the test, as described in its README.

MCP usage

The catalog contains 82 tools; chapters 25 and 49 list the three SQL tools. **Share IDE recordings with MCP** and per-tool permissions apply. Reading/comparing does not execute new Java.

- `repl_recording_sql` returns `events`, `statistics`, `nextOffset`. Use `sql-id`, `text-offset`, `text-limit` for an SQL text; `nextTextOffset` is in the event row. Text defaults to 512, maximum 2048 characters.
- `repl_recording_findings` returns `findings`, root, count, duration, source and example SQL/parent IDs. SQL text is shortened; use the SQL tool for the full captured template.
- `repl_recording_pin` preserves the call's SQL subtree together with its values for that client.

- `repl_recording_sql_compare` returns `sqlBefore`, `sqlAfter`, `sqlDelta`. Supply exactly one of `before` / `reference` plus `after`. Normal `repl_recording_compare` also includes these statistics.
- `recording` and `view` protect paging. SQL changes also update view. With `partial=true`, `pending` or `dropped`, do not claim N+1 has conclusively disappeared.

Task for Claude: "Read current SQL events and suspected N+1 groups. Identify the request root and application caller of the repeated SQL. Pin the call subtree; after a fix and new recording, compare the same request. Do not trigger business operations without a separate request."

Boundaries and retained data

Synchronous Spring MVC and JDBC/JPA are supported. An HTTP root is created once the request reaches a selected class. Subsequent SQL on that dispatcher thread also attaches there. HTTP method/path may be recorded; query string, headers and body are not part of the HTTP root.

Normalization removes SQL string/numeric literals and comments. Parameter values, ResultSet content and database passwords are not collected; errors record exception class. Other parts of a recording can contain application data/source, so it is not generally anonymized.

Separate budgets apply: **1000 JDBC events / 1.5 million encoded characters**, **200 Java calls**, **32 MiB value previews**. Limited, missing or pending evidence is marked. SQL for a PreparedStatement created before late attach may be unavailable.

42. Hibernate in recorded call flows

Version 0.22 links Hibernate 6.6 ORM events to recordings. See which entity/association lazy initialization triggered a SELECT, whether flushes or writes occurred and whether cache served a query. Existing StatementInspector and event listeners are preserved. The plugin does not enable or reset global Hibernate Statistics counters.

Enable and follow the flow

1. After installing the updated plugin, restart both IDEA and the target application so the new agent is loaded.
2. **Tap / Trace > Recorded calls > New recording...**: select controller/service/repository classes. Alongside JDBC/SQL, enable **Record Hibernate 6.6 entity / session events**.
3. Trigger the operation and stop recording. Select the request using **Root**.
4. Click an ORM node to open **Hibernate** details. Graph and timeline use the same recorded events; related JDBC nodes are children of the ORM operation.

Example: service call > `Order.customer` lazy initialization > `select ... where id=?`. Java parent, ORM parent, root, thread and session have separate IDs. A session token is not an entity identifier or a reusable object handle.

Buttons and filters

Label / location	Purpose
Hibernate details tab	Entity loads, associations, flush, dirty checking, cache and transaction events within the current Root filter.
Entity / relationship / source	Search entity/association names, description, source class and method. Apply with Filter or Enter.
All events / Findings / event type	Show all events, only events associated with findings, or one event type.
Select an event	Show session, source location, timing, error type, finding explanation and related SQL. Details are scrollable.
Open originating call	Select the recorded Java caller with source and captured input/result. Does not execute code.
Open entity mapping	Open the entity's current project source, which may differ from recorded code. Missing source is reported.
Pin Hibernate baseline	Preserve ORM counters for the current Root filter; survives a new recording.
Clear baseline	Clear the Hibernate reference. SQL baseline and Java Pin reference are separate.
Save / Open recording	Store ORM evidence with SQL and Java calls; earlier file versions remain readable.

Which events are available?

- **ENTITY_LOAD / INSERT / UPDATE / DELETE:** entity operations. Update shows changed property names, not their values.

- **LAZY_ENTITY / COLLECTION_INIT / LAZY_ATTRIBUTE:** proxy, persistent collection or bytecode-enhanced attribute initialization. Association names may be `Order.customer`; ambiguous evidence is labeled unknown or multiple associations.
- **QUERY:** normalized HQL. Linked SQL events show actual database executions.
- **FLUSH / AUTO_FLUSH / DIRTY_CHECK:** flush, automatic flush checks and dirty checking. AUTO_FLUSH details indicate whether a flush was actually required.
- **TRANSACTION_BEGIN / COMMIT / ROLLBACK**, plus **TRANSACTION:** transaction boundaries and completion outcome associated with a session.
- **CACHE_HIT / MISS / PUT:** L2 cache events. **QUERY_CACHE_HIT / MISS / PUT:** query-cache events. The plugin does not enable caches or change the cache provider configuration.

Entity-load count is not SQL count: one query can load multiple entities and a cache hit can create a managed entity. Lazy initialization need not execute SQL either. No complete L1 persistence-context hit/miss counter is claimed. Hibernate duration may include nested JDBC time; do not sum overlapping durations.

N+1 and lazy loads during response rendering

suspected-n-plus-one requires repeated lazy initializations and actual SELECTs within the same root, association and source location. New recording sets the threshold, default 5. Lazy operations served from cache without SQL do not produce this finding on their own.

lazy-during-response identifies lazy loading during synchronous Spring MVC return-value handling or later rendering, such as JSON serialization. It is a separate finding, not every operation after a controller returns. Details explain it and list concrete SQL IDs. Both findings are investigative hints, not automatic proof of a faulty fetch strategy.

Compare the same request with equivalent input, profiles and cache state. **Pin Hibernate baseline** compares entity/lazy/flush/cache counters; SQL baseline supplies actual JDBC counts. A fetch-join fix might reduce 1+5 SELECTs to one while retaining the result.

Inspector: browsing without lazy loading

Inspector recognizes Hibernate proxies, persistent collections and observed entities. It shows initialization and observable managed/attached/detached state. It does not open uninitialized proxy/collection contents or call size, iterator or entity getters. Already initialized values expose loaded fields or the collection's backing storage.

An enhanced entity's unloaded field displays **Unfetched Hibernate attribute; not read**. A default null is not presented as actual loaded data. Unsupported or unobserved entities may have unknown state; the UI does not invent managed status. Explicit REPL getter calls remain possible and can trigger real lazy loading.

CASE limits and JUnit export

Cases / Reload > Options provides four optional fields:

Field	Limit
Maximum entity loads	Number of ENTITY_LOAD events.
Maximum Hibernate flushes	FLUSH plus required=true AUTO_FLUSH events.
Maximum lazy initializations	Proxy, collection and enhanced-attribute initializations together.
Maximum lazy loads during response handling	Those lazy operations associated with synchronous MVC response handling.

Blank means no assertion; zero is valid and the range is 0-1000000. Measurement covers Code and Result expression, excluding input loading, Imports, Setup, Cleanup and comparison. Exceeding a bound means FAILED. Partial evidence means INCONCLUSIVE unless a known excess already proves FAILED. Missing/unsupported adapters cannot silently produce passing zero counts.

JUnit ZIP retains these limits and includes the matching `runtime/sb-repl-agent.jar`. The ORM test JVM requires:

```
-javaagent:/abs/path/runtime/sb-repl-agent.jar=port=0
```

For Maven add it to Surefire argLine; for Gradle use the Test task's jvmArgs, preserving existing options such as JaCoCo. Configuring only the build daemon is insufficient. Hibernate 6.6 and a previously opened session are required; JPA/Spring initialization usually creates one. If necessary, open and close a session in CASE Setup. The export README describes these requirements. The agent also starts a local authenticated development endpoint. IDEA is unnecessary; SQL-only exports still run without an agent.

MCP and limits

Chapter 25 lists three tools: `repl_recording_hibernate`, `repl_recording_hibernate_findings`, `repl_recording_hibernate_compare`. The complete catalog has **82 tools**. Existing **Share IDE recordings with MCP**, tool allowlist, redaction and audit apply. Reading/comparing executes no new application code. Pin preserves ORM and SQL subtrees; statistics compare through hibernateBefore/After/Delta.

The tested adapter targets **Hibernate 6.6.29.Final**, on Java 17/21. Complete measurement is not claimed for other Hibernate versions. Synchronous session/MVC/JDBC and supported linked worker threads are observed; reactive/R2DBC, StatelessSession and database execution-plan/lock internals are outside this adapter. Events before late attach cannot be recovered.

Hibernate has an independent budget of **1000 events / 1.5 million encoded characters** and 64 ORM parent levels. With omitted/dropped, pending, unavailable or partial evidence, counters cannot prove an upper bound for the complete run. Recordings show observed paths associated with selected classes, not a complete application profile.

43. Recorded call to runnable CASE

Location: Tap / Trace > Recorded calls. The 0.23 workflow turns a completed call into an editable reproduction without running it again.

1. Choose **New recording...** and select the application classes.
2. Enable **Capture replay DATA** before triggering the real request. This is opt-in because snapshot serializers can invoke getters and add work to the application thread.
3. Select a completed Java call and choose **Create CASE from call...**, or right-click its graph node.
4. Enter an unused CASE name and select the Spring bean. A static method needs no bean. Ambiguous receivers require a choice.
5. Open **Cases / Reload**, refresh and load the new CASE. Review Code, Result expression, input type, expected outcome, Observed classes and SQL/ORM limits before explicit execution.

What is saved?

The operation creates `name-input`, `name-expected` and `name`. Input contains arg0, arg1 and subsequent arguments captured at method entry. The outcome is captured at exit. Later mutations do not alter these detached values. Generated Java converts each argument to the declared public type and calls the selected bean, retaining its proxy advice. JDK proxies use a public interface; CGLIB proxies use the application type.

Exceptions create type/message expectations. Available, complete SQL and Hibernate evidence can initialize budgets. Incomplete asynchronous or truncated evidence cannot supply a proven upper bound. These limits are a starting point to review, not automatically approved behavior.

Boundaries

Full capture has a separate **2 MiB input/result and 32 MiB recording budget**. The existing display previews and DATA storage limit remain separate. Public, accessible signatures and serializable data are required. Future/CompletionStage results need capture of the completed value inside the worker. Framework infrastructure, unsupported types and exceeded budgets produce a visible unavailable result instead of a misleading replay.

Full replay DATA lives in the current runtime recording. The portable `.sbrepl-recording` file stores display evidence, not this full DATA. Create the CASE before replacing/resetting/disconnecting the recording; the resulting persistent DATA and CASE can then be exported through existing reproduction/workspace/JUnit functions.

Generated conversion uses a Jackson ObjectMapper with available modules. Application-specific mixins, custom codecs, security/tenant context and time-dependent behavior need explicit review or setup. This feature prepares a reproduction; it does not restore an entire application execution environment.

44. Edit DATA copies and create variants

Location: Snapshots > Saved > **Edit DATA copy**. Select a DATA snapshot first. Inspector also has **Edit DATA copy...**: it first freezes a new original DATA from the selected live value, then opens the same editor.

Control	Behavior
New DATA name	Must be different and unused. Saving never overwrites the source DATA.
Restore type	Declared Java type used to validate the edited payload.
JSON editor	Formatted, detached payload. Changing it does not modify a live object.
Validate type	Parse JSON and attempt typed deserialization. Constructors/custom deserializers can run application code.
OK	Validate again and save a new DATA copy. A changed source checksum rejects a stale edit.
Cancel	Close the editor; it does not undo a save that was already dispatched.

Interactive editing supports **2 MiB**. Larger DATA should be projected to a smaller DTO or exported. Validation checks the selected declared type; a generic Map is not validation of every nested business DTO. The copy records the source name and source version.

Parameter variations

In **Cases / Reload**, select one saved CASE and click **Create variants...**. Supply a new CASE name and 1-20 distinct DATA names, one per line. The new CASE receives one parameter row per input. It initially reuses the original expected DATA for every row; review and adjust the Parameters tab before running. Creating variants runs no case and does not change the source definition.

Example: freeze an order, make copies with quantity 0, 1 and 100, then build a three-row CASE. Decide the expected result or exception for each row before executing real Spring beans.

45. Bean Explorer

Location: Tools > Beans. **Workbook > Tools > Bean explorer** opens it. This is distinct from Insert bean, which remains a quick declaration helper.

Control	Purpose
Name / type + Search / refresh	Search definitions and existing singletons without instantiating lazy beans. Enter applies the search.
Previous / Next	Read another page of 100 beans.
Select bean	Show scope, primary/lazy flags, qualifiers, aliases, factory/resource and proxy/target information when available.
Dependencies	Resolved dependencies and dependents. Double-click an entry to follow it. Uninitialized dependencies may not yet appear.
Method selector	Choose a supported public method by exact JVM signature, including overloads.
Prepare method call...	Choose compatible DATA for each argument and insert generated Java into the workbook. Does not invoke the bean or deserialize the DATA.

DATA suggestions use declared types and conservative raw-type assignability. They are candidates, not proof that every field can be converted. If no candidate exists, freeze/edit a DATA snapshot with the required type first. Generic signatures that cannot be named safely are unavailable.

Review the generated code, use Check code, then run it explicitly. This last step deserializes values and can instantiate a lazy bean or trigger real application effects. Proxy metadata is observational; the Explorer does not replace or bypass Spring proxies.

46. Watches and field changes

Location: Workbook's result area > **Watches**. **Watch result** opens the pin dialog with `last1`; Pin watch can use another session variable.

Control / tab	Meaning
Pin watch...	Add a variable/field/index/key path such as last1.items[0].total or last1.items.size(). Adding alone does not sample.
Allow Java expression / method calls	Explicitly allow arbitrary Java expressions for this watch. These can execute application code after each REPL evaluation. Disabled by default.
Refresh values	Sample explicitly under the current execution settings. There is no timer evaluating watches.
Remove	Remove the selected watch from this session.
Current / Previous	Expand the last two detached display captures.
Changed fields	Compare bounded field paths and values.

Watches sample after an explicit REPL evaluation and do not replace last1/last2 or the main result handle. Default paths use fields and safe JDK container access; application getters are not called and unfetched Hibernate relationships stay unfetched.

States are FIRST, CHANGED, UNCHANGED, PARTIAL and ERROR. PARTIAL, limits and unreadable fields cannot establish equality. A watch error may leave the previous successful value visible with ERROR; it is not a fresh result. At most 20 watches, with bounded before/after previews, are retained per session. Reset/disconnect discards them. IDE and MCP watches belong to their separate sessions.

47. Async flow and transaction boundaries

Location: New recording... > **Link Executor / @Async / CompletableFuture tasks**. The checkbox is enabled by default in the IDE recording dialog; the MCP start tool requires `async="true"` explicitly. Restart the target application with the updated agent before using this capability.

An **Async handoff** node connects a recorded submitter with work on a supported executor. Dashed edges distinguish task boundaries. The node shows queueWaitMs, submission time, task type, cancellation-at-start and whether a transaction was active on the submitting and executing threads. Select the worker's Java child to open its source and values; the synthetic boundary itself has no application source.

The adapter observes ThreadPoolExecutor, ForkJoinPool task execution, Spring ThreadPoolTaskExecutor and SimpleAsyncTaskExecutor. Packaged-agent tests cover Java 17/21, CompletableFuture continuations and Spring @Async. SQL/ORM in a linked worker uses the same root while retaining its actual thread identity.

Only recording identity is propagated. The REPL does not copy transactions, security, tenant data, MDC or arbitrary ThreadLocals. A parent ROLLBACK transaction therefore does not guarantee rollback of worker DB writes. Non-async continuation stages may run inline; custom executors, virtual-thread executors, Reactor and remote queues are not generally covered.

Task capture is bounded to 1024 pending identities with a five-minute lifetime. Ambiguous concurrent reuse of the same Runnable, expired/collected tasks and overflow are reported as unlinked/dropped evidence. Queued work keeps observations incomplete; stopping a recording abandons unstarted links. The graph does not invent missing ancestry. The boundary describes task execution, not guaranteed successful completion of every Future; exceptions captured inside a Future can require reading the Java child outcome.

Recording file version 4 preserves explicit cross-thread handoffs and async availability/pending/drop metadata. Previous recording formats remain readable. Neither task capture nor reopening a file resumes a suspended stack.

48. HotSwap and affected CASE review

Location: Cases / Reload > **Reload + affected CASEs...**. Select the complete Java source to reload first, as for Reload + run selected.

The plugin parses the selected source without executing it, compares its class names with each CASE's **Observed classes**, and opens a review dialog. CASEs with an observed match start checked. Unknown coverage and cases with no observed match are labeled separately; either may still be affected. Choose up to 20 cases before proceeding.

The chosen source is reloaded first. Only a successful HotSwap permits the selected saved CASEs to run. Unsupported structural changes, compile errors and cancellation stop the workflow. Changes in the CASE editor must be saved separately; the workflow runs saved definitions. There is no automatic business-code rerun on every file save.

Compare with the previous run

The report includes before/after outcome, result fingerprint, exception, SQL/ORM counters and duration where evidence exists. First execution establishes the session baseline. Changing a CASE, DATA version, Spring context or execution settings makes the old baseline incomparable. HotSwap can then be evaluated against an otherwise unchanged baseline.

Missing/partial counters are UNKNOWN. Result fingerprints use canonical detached JSON; they do not compare live object identity. One duration measurement is diagnostic context, not proof of a performance regression. Assertions and explicit budgets decide PASSED/FAILED. Baselines and detailed results are session-local and the most recent 20 CASE results are retained.

49. MCP workflow tools, events and tasks

The catalog now contains **82 tools**. These 16 additions expose the new workflows. Existing allowlists, separate execution, snapshot-write, capture, CASE-run, HotSwap and recording-sharing settings continue to apply.

Tool	Action
repl_bean_search	Search bean names/types, 100 rows per page.
repl_bean_info	Definition, proxy, dependencies and method signatures.
repl_bean_compatible_data	Candidate DATA for one method argument.
repl_bean_prepare	Prepare Java only; review before eval.
repl_snapshot_edit_read	Read detached payload/type/source checksum.
repl_snapshot_edit_validate	Attempt typed deserialization; requires execution.
repl_snapshot_edit_copy	Save to a new name; requires execution and snapshot writes.
repl_case_variants	Clone a parameterized CASE without running it.
repl_case_affected	Suggest cases from observed classes; no reload/run.
repl_watch_add	Pin a session watch; allow-java is explicitly opt-in.
repl_watch_list	Watch identities and last state.
repl_watch_get	Last before/after captures and differences.
repl_watch_remove	Remove a session watch.
repl_watch_refresh	Explicit sampling under the MCP execution policy.
repl_recording_case_info	Check full DATA availability in the live IDE recording.
repl_recording_case_create	Create persistent DATA/CASE from captured input/outcome.

`repl_recording_start` adds string booleans `capture-data` and `async`, both false by default for MCP. CASE creation needs execution, snapshot writes and Share IDE recordings. Creating a CASE never invokes its method. The independent MCP evaluator can subsequently load/run the saved CASE under its own policy and CASE-run permission.

Event subscriptions

Standard MCP resources expose **repl://session/events**. A client lists/reads it and calls resources/subscribe. Authenticated GET /mcp with the same session and Accept: text/event-stream carries resources/updated invalidations and task status notifications. The client reads the resource to obtain metadata. No Java code or captured values are broadcast.

Runtime journals report capture completion, context changes, execution/CASE completion and recorded calls. When recording sharing is enabled, changes to the shared IDE recording also invalidate the resource. The server polls metadata internally once per second; the AI does not need to poll tools continuously. Resources/unsubscribe stops that subscription. Each client has one stream; short reconnects can use Last-Event-ID.

The runtime keeps 128 events; the SSE replay buffer keeps 256 notifications. Gaps and expired replay are explicit: refresh the resource/task state and reconnect without an old event ID. Sessions and events remain local, authenticated and isolated. Event subscriptions do not extend a session forever without client requests.

Identifiable long-running work

With MCP 2025-11-25, eval, CASE run/batch, reload and watch refresh accept an optional task field on tools/call. The initial response contains a working task ID. Use tasks/get or notifications/tasks/status, then tasks/result for the original tool result. Tasks/list shows this client's retained work. Tasks/cancel requests cooperative interruption and permanently marks that task cancelled; it does not undo application effects or release a still-running evaluator immediately.

At most 20 tasks are retained per session. TTL is bounded and reported by the server; results are in memory and expire with the session/server. A transport failure retains the task's failure report but disables further execution on that session. Never resubmit an uncertain operation automatically. Normal synchronous tool calls remain available for clients without task support; actual support depends on the MCP host.

MCP tasks specification describes this experimental protocol facility. MCP resources and Streamable HTTP define subscription and transport behavior.

50. Data handling and font licenses

The plugin does not send telemetry, source code or runtime values to the publisher. Local workspaces and snapshots can contain application data. The optional AI panel sends the exact reviewed prompt to the configured provider only after **Send reviewed prompt**. An authenticated local MCP client can read permitted runtime data and may forward it to its own provider.

Workbook recovery checkpoints are in the IDE system directory under `java-repl-workspaces`. Persistent snapshots are in the application's `~/.java-repl-snapshots`; redacted audit logs are in `~/.java-repl-audit` unless overridden. Snapshot and workspace contents have no automatic expiry. Uninstalling the plugin does not automatically delete those files or PasswordSafe entries.

Use **Tools > Snapshots > Saved > Delete** to delete a snapshot and its versions. **Workbook > Workspace > Clear history** clears snippet history. Delete exported files and recovery/audit folders manually after closing the project and application when you want to remove those retained copies. Removing secrets from IDE PasswordSafe invalidates dependent client configurations. The complete data-handling document covers retention, sharing and contact details.

Embedded handbook fonts

These PDFs use DejaVu Sans, Sans Bold and Sans Mono subset fonts. Their complete notices and permissions follow. Other bundled third-party terms are retained in their original JARs and summarized in THIRD-PARTY-NOTICES.txt.

The following software may be included in this product: Deja Vu fonts. Use of any of this software is governed by the terms of the license below:

Fonts are (c) Bitstream (see below). DejaVu changes are in public domain. Glyphs imported from Arev fonts are (c) Tavmjong Bah (see below)

Bitstream Vera Fonts Copyright

Copyright (c) 2003 by Bitstream, Inc. All Rights Reserved. Bitstream Vera is a trademark of Bitstream, Inc.

Permission is hereby granted, free of charge, to any person obtaining a copy of the fonts accompanying this license ("Fonts") and associated documentation files (the "Font Software"), to reproduce and distribute the Font Software, including without limitation the rights to use, copy, merge, publish, distribute, and/or sell copies of the Font Software, and to permit persons to whom the Font Software is furnished to do so, subject to the following conditions:

The above copyright and trademark notices and this permission notice shall be included in all copies of one or more of the Font Software typefaces.

The Font Software may be modified, altered, or added to, and in particular the designs of glyphs or characters in the Fonts may be modified and additional glyphs or characters may be added to the Fonts, only if the fonts are renamed to names not containing either the words "Bitstream" or the word "Vera".

This License becomes null and void to the extent applicable to Fonts or Font Software that has been modified and is distributed under the "Bitstream Vera" names.

The Font Software may be sold as part of a larger software package but no copy of one or more of the Font Software typefaces may be sold by itself.

THE FONT SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT OF COPYRIGHT, PATENT, TRADEMARK, OR OTHER

RIGHT. IN NO EVENT SHALL BITSTREAM OR THE GNOME FOUNDATION BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, INCLUDING ANY GENERAL, SPECIAL, INDIRECT, INCIDENTAL, OR CONSEQUENTIAL DAMAGES, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF THE USE OR INABILITY TO USE THE FONT SOFTWARE OR FROM OTHER DEALINGS IN THE FONT SOFTWARE.

Except as contained in this notice, the names of Gnome, the Gnome Foundation, and Bitstream Inc., shall not be used in advertising or otherwise to promote the sale, use or other dealings in this Font Software without prior written authorization from the Gnome Foundation or Bitstream Inc., respectively. For further information, contact: fonts at gnome dot org.

Arev Fonts Copyright

Copyright (c) 2006 by Tavmjong Bah. All Rights Reserved.

Permission is hereby granted, free of charge, to any person obtaining a copy of the fonts accompanying this license ("Fonts") and associated documentation files (the "Font Software"), to reproduce and distribute the modifications to the Bitstream Vera Font Software, including without limitation the rights to use, copy, merge, publish, distribute, and/or sell copies of the Font Software, and to permit persons to whom the Font Software is furnished to do so, subject to the following conditions:

The above copyright and trademark notices and this permission notice shall be included in all copies of one or more of the Font Software typefaces.

The Font Software may be modified, altered, or added to, and in particular the designs of glyphs or characters in the Fonts may be modified and additional glyphs or characters may be added to the Fonts, only if the fonts are renamed to names not containing either the words "Tavmjong Bah" or the word "Arev".

This License becomes null and void to the extent applicable to Fonts or Font Software that has been modified and is distributed under the "Tavmjong Bah Arev" names.

The Font Software may be sold as part of a larger software package but no copy of one or more of the Font Software typefaces may be sold by itself.

THE FONT SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT OF COPYRIGHT, PATENT, TRADEMARK, OR OTHER RIGHT. IN NO EVENT SHALL TAVMJONG BAH BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, INCLUDING ANY GENERAL, SPECIAL, INDIRECT, INCIDENTAL, OR CONSEQUENTIAL DAMAGES, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF THE USE OR INABILITY TO USE THE FONT SOFTWARE OR FROM OTHER DEALINGS IN THE FONT SOFTWARE.

Except as contained in this notice, the name of Tavmjong Bah shall not be used in advertising or otherwise to promote the sale, use or other dealings in this Font Software without prior written authorization from Tavmjong Bah. For further information, contact: tavmjong @ free . fr.